SiFive iommu22_sv48_ats_max Manual

24G1.02.00

SiFive iommu22_sv48_ats_max Manual

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 About this Document

This document describes the functionality of the SiFive IOMMU-22 address translation engine (previously known as PLATEv2). The SiFive IOMMU-22 is compliant to the RISC-V IOMMU Specification (Version 1.0.0, July 2023). This document should be read in conjunction with the RISC-V IOMMU Specification: https://github.com/riscv-non-isa/riscv-iommu/releases/download/v1.0.0/riscv-iommu.pdf.

IOMMU-22 also supports DTI-ATSv2 as defined in the Arm® AMBA® Distributed Translation Interface (DTI) Protocol Specification (ARM IHI 0088E.b).

Refer to the `docs/systemip_configuration.txt` file for a comprehensive summary of the iommu22_sv48_ats_max configuration.

1.2 Overview

IOMMU is an address translation engine integrated with an IO bridge between IO device(s) and the system interconnect. Traditionally, an IO device can include a GPU for graphics, storage controllers, NIC or IO accelerators (such as encryption accelerators or DSPs), which might have a DMA interface to the system. The primary function of IOMMU is to translate device virtual addresses to physical addresses for inbound device requests, and to perform memory protection for such requests. Virtualization of interrupts (interrupt remapping and virtualizing) is not addressed in this document.

An example of IOMMU integration in DMA traffic is depicted in Figure 1.

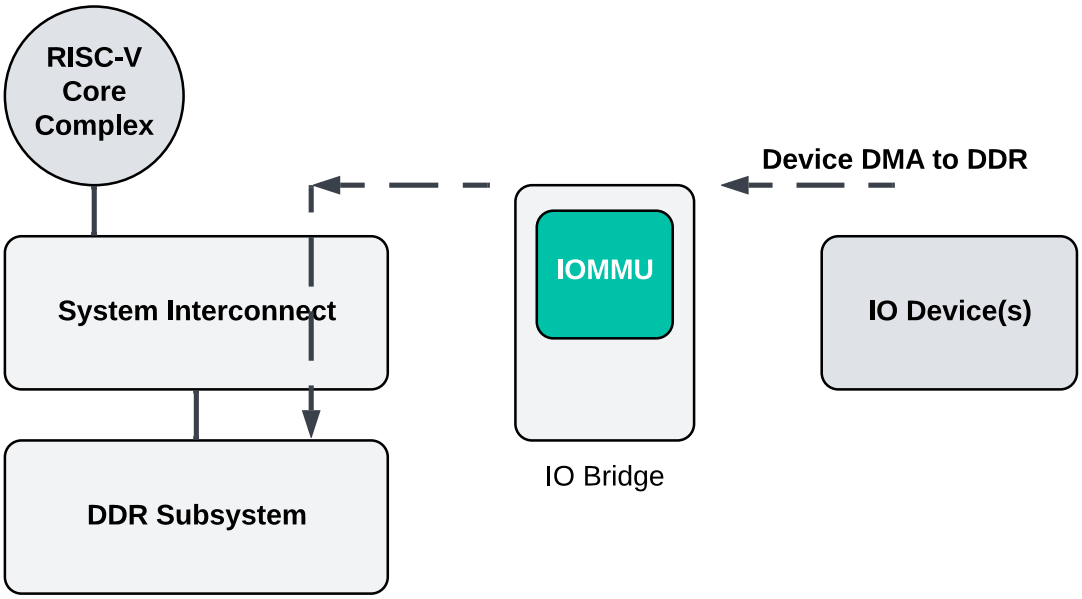


Figure 1: Example of IOMMU Integration in DMA Traffic

IOMMU is integrated in an IO bridge or shell in order to comply with the SoC requirements. The role of this IO bridge may differ from SoC to SoC, but it may have to handle the request reordering, error handling, and/or specific attribute management. The IO bridge is not described in this document.

In Figure 1 above, device DMA requests to the memory subsystem, referred to as inbound transactions, can be processed by IOMMU. Outbound transactions, from the RISC-V core to the IO device, are not managed by IOMMU as the address of the transaction is already physical (translated by the hart's MMU).

For IOMMU to perform address translations, it must differentiate between the various IO devices to select the proper translation rules. Inbound requests have a hardware context identifier along with the request (address, read/write, attribute) to identify a translation entry.

This identifier is referred to as a **HardContextID (HCID)**. This identifier must be unique to a device (or a process within the device). The HCID consists of two components: the **DeviceID**, which is a hardware identifier of the IO device (or a portion of the device), and the **ProcessID**, which is the process context identifier when multiple contexts are supported by the device. Each device can provide several HCIDs to IOMMU based on the transaction. Once a device provides an HCID to IOMMU, that HCID cannot be used by any other device connected to the same IOMMU instance.

The IOMMU associates a software process with a HCID by binding a **SoftContextID (SCID)** to a HCID using a context table. A SCID is a representation of a software process (e.g., using an ASID).

A single IOMMU may be used to perform address translation from one device or several devices, There may also be several IOMMU instances in an SoC, with each one translating addresses for a set of IO devices.

The IOMMU supports functions similar to the MMU of a RISC-V hart.

IOMMU supports single stage translation: the original Virtual Address VA (or Supervisor Virtual Address) is converted in the first stage using Supervisor Page tables (gathering all Page Table Entries) into a System Physical Address (or Supervisor Physical Address).

Throughout the document, the original device virtual address or the supervisor virtual address will be referred to as Virtual Address (VA) and the system physical address or supervisor physical address will be referred to as Physical Address (PA).

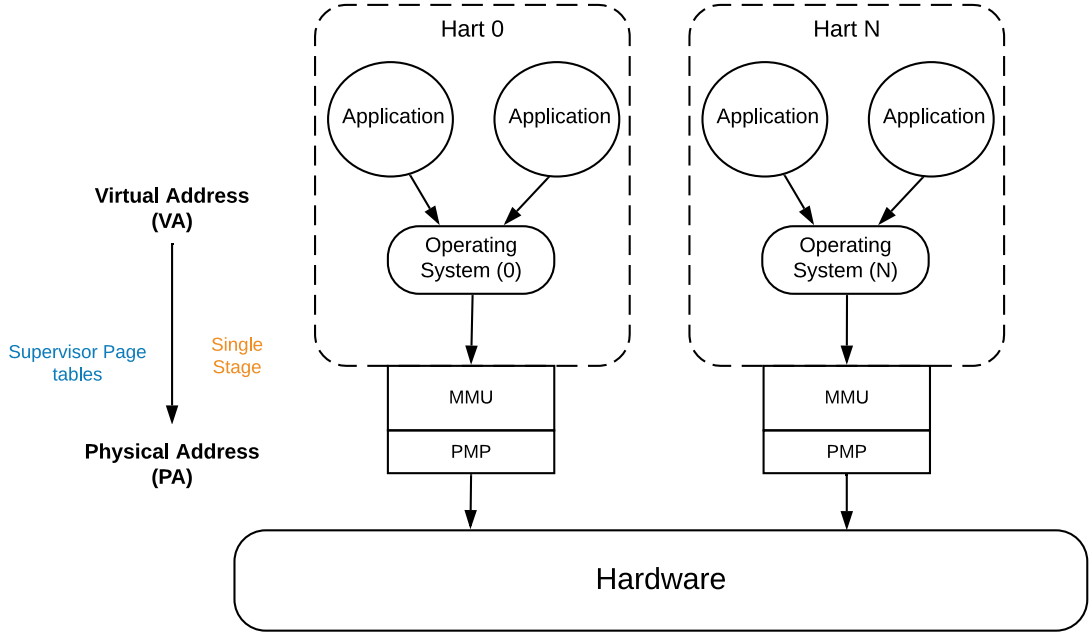


Figure 2: Translation in a Non-Virtualized System

IOMMU also supports two stages of translation: the Guest Virtual Address (GVA) is converted in the first stage by Virtual Supervisor Page tables into a Guest Physical Address (GPA), and the Guest Physical Address is then converted in the second stage by Hypervisor Guest Page tables into a System Physical Address (SPA). It is capable of performing all possible translations supported by the cores (single stage, VS-stage, G-stage, and nested).

MSI address translation using the G-stage page tables can be sent to the proper interrupt file in the IMSIC. PCIe standard is supported along with Address Translation Services and Page Request Interface. This second level of IOMMU-22 does not support single level MSI page tables, memory resident interrupt file, and atomic operations.

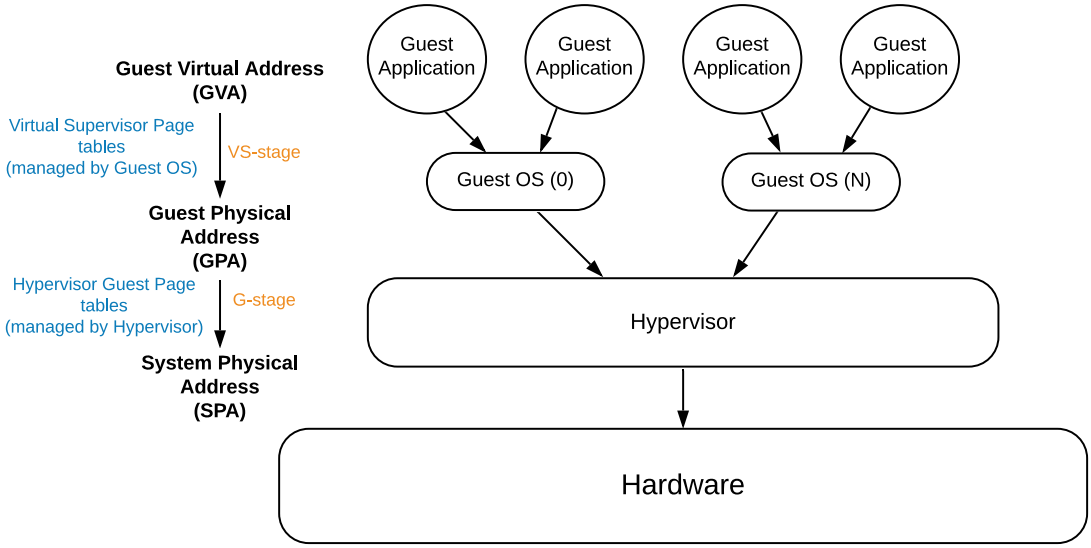


Figure 3: Translation in a Virtualized System

1.3 Features

IOMMU-22 supports the following features:

- Supports RISC-V paging schemes: Sv39 and Sv48
- IO DMA access permission checking (memory protection) using Page Table Entries
- IO DMA virtual address translation using Page Table Entries
- Use of CPU page tables or separate IO device page table for DMA translation
- Supports single stage translation (VA to PA)
- Supports virtualization with dual stage translation: VS-stage (stage 1), G-stage (stage 2), and nested translation (stage 1 and stage 2 translation)
- Up to 36-bit HardContextID
  - Up to 16-bit DeviceID
  - Up to 20-bit ProcessID
- Up to 36-bit SoftContextID
  - Up to 16-bit GSCID (which identifies the VM ~VMID)
  - Up to 20-bit PSCID (which identifies the address space ~ASID)
- Caching of context table entries. Full table is set in System memory
- Memory-based queues for exchanging Commands and Error Records information between IOMMU and the harts
- Svnapot and Svpbmt extensions

- PCIe ATS and PRI when used with a PCI Express Root Complex

Optional features in the RISC-V IOMMU Specification not supported by IOMMU-22:

- Single level MSI Page Table and MRIF support
- Atomic updates of MRIF and A/D bits in PTE
- Up to 24-bit DeviceID for PCIe Gen6 support
- RV32 support (Sv32 or Sv32x4) and Sv57 support
- Big- or mixed- endian

1.4 Use Cases

IOMMU provides the following significant system-level functionalities:

1. Protect the operating system from bad memory accesses from errant devices
2. Support 32-bit devices in 64-bit environment
3. Hardware support for IO device Virtualization: IO devices assignment to Virtual Machines or Guest Operating Systems

1.4.1 Operating System Protection

In the absence of IOMMU, a device driver must program IO devices with Physical Addresses, which implies that DMA-ed data from an IO device could be written to privileged memory. This can be caused by hardware, device driver bugs, or malicious software. The consequence can be a system crash. Malicious software attacks can come from IO peripherals that make read/write accesses to the system memory or even to embedded memory in other peripherals through DMA transactions. Such read/write can cause leakage of sensitive data. The integration of IOMMU offers a mechanism to protect the OS from these software attacks and from buggy device drivers because IOMMU ensures that privileged memory of the OS is isolated from a device.

1.4.2 32-bit Devices in 64-bit Environment

In the absence of IOMMU, legacy 32-bit devices cannot access the entire 64-bit system memory range. This forces the operating system to ensure that when this 32-bit IO device is used, buffers in the first 4GB memory region are allocated and then contents of this buffer are copied to the final destination in the system memory to be consumed afterwards. The integration of IOMMU offers a simple mechanism for the DMA to directly access any address space in the system (with appropriate access permission) and thereby improves system performance.

1.4.3 IO Device Virtualization

The presence of IOMMU in an SoC allows software to configure IO device DMA with virtual addresses instead of physical addresses.

When an IO device is assigned to a Guest OS, the device can be configured directly using the VA address space and the Host configures IOMMU to perform single stage VA-to-PA translation.

In a system with the Hypervisor Extension, the G-stage translation (or second stage of translation) is intended to virtualize IO device DMA to the Guest OS address space. IO Devices can be assigned to Guest OS which can directly set the DMA with its Guest Physical Addresses (GPA). The Hypervisor or Host OS will set up and configure IOMMU to perform GPA-to-SPA translation using G-stage page tables. Interrupts are expected to be handled by the Host or Hypervisor software.

The two stages of translation (VS-stage and G-stage) are intended to be used by a software entity to provide isolation or translation to buffers within the entity, for example DMA isolation within an OS. An incoming address to IOMMU is logically translated from GVA to GPA in VS-stage, then the GPA is translated in G-stage to the output SPA.

DMA devices generally do not handle page faults well. The system must consider pinning the IO pages to avoid page fault. A guest would need to request pinned pages from the hypervisor.

1.5 Integration Examples

The following example describes a typical SoC with RISC-V hart(s). The SoC incorporates memory controllers for DRAM storage and several IO devices. This SoC also incorporates two instances of IOMMU. The first instance interfaces one IO Device A to the system fabric and the second instance interfaces several clients (IO Devices B, C and D) to the system fabric.

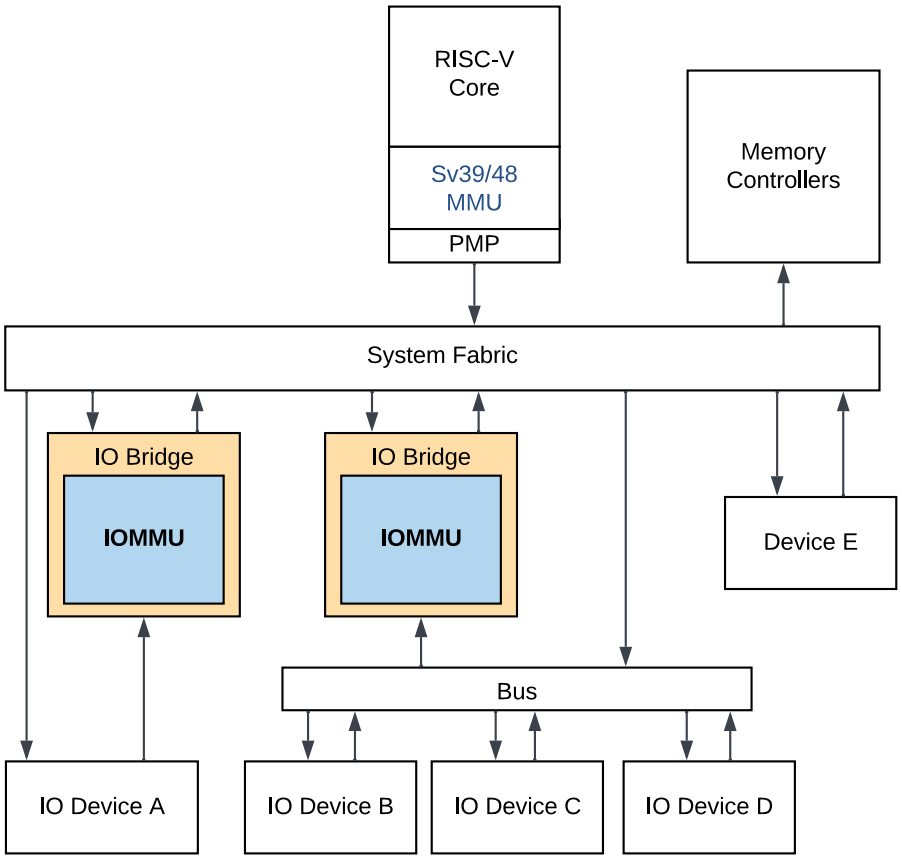


Figure 4: IOMMU Integration Example

Device drivers running on the hart can configure the IO device's DMA engine. IO devices can perform DMA transactions using Virtual Addresses (VA, GVA, or GPA) or any other bus addresses that IOMMU will translate into Physical Addresses (PA). IOMMU has a host interface used as a programming interface for software running on a hart to configure and maintain the translation and protection tables. IOMMU's page table walker can access system memory through its master port.

Figure 5 focuses on IOMMU's integration into a PCIe system where the Root Port is connected to several endpoints with or without ATC support. An optional ATS interface would communicate between IOMMU's translation tables and the Root Port to handle the endpoint's translation cache.

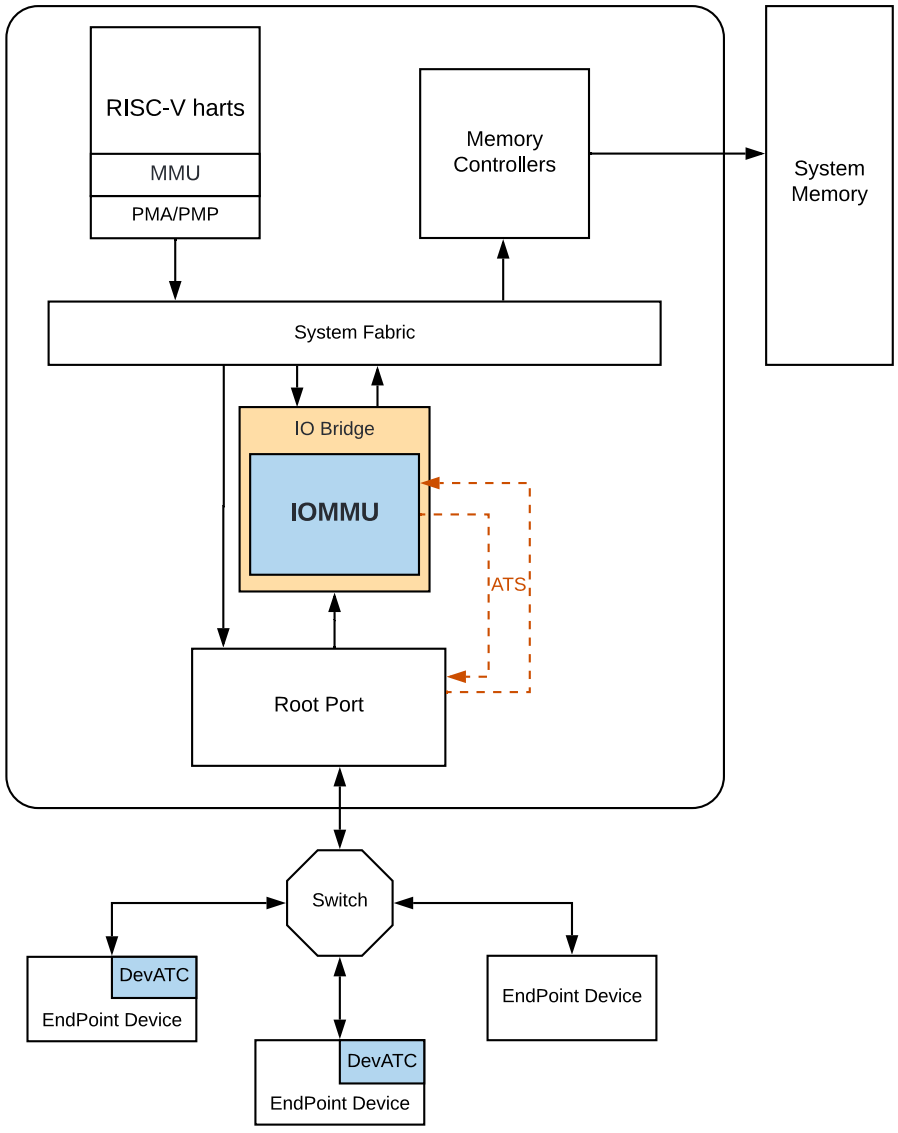


Figure 5: IOMMU in a PCIe System

In this SoC, IOMMU performs address translation from each endpoint's Virtual Address to a Physical Address. IOMMU can also translate MSI or MSI-x addresses to whatever the translation table points to. IOMMU handles MSIs as regular inbound transactions. A unique HardContextID per endpoint's function is essential for IOMMU to perform the proper address translation.

Note

One SoC can have several PCIe Controllers handled by a single IOMMU instance. The HardContextID provided to IOMMU must be constructed such that each endpoint has a unique identifier (for instance, MSB of HardContextID could be the Root Segment ID).

Chapter 2

Architecture

2.1 IOMMU Interfaces

This section focuses on IOMMU's interfaces, which are depicted in Figure 6.

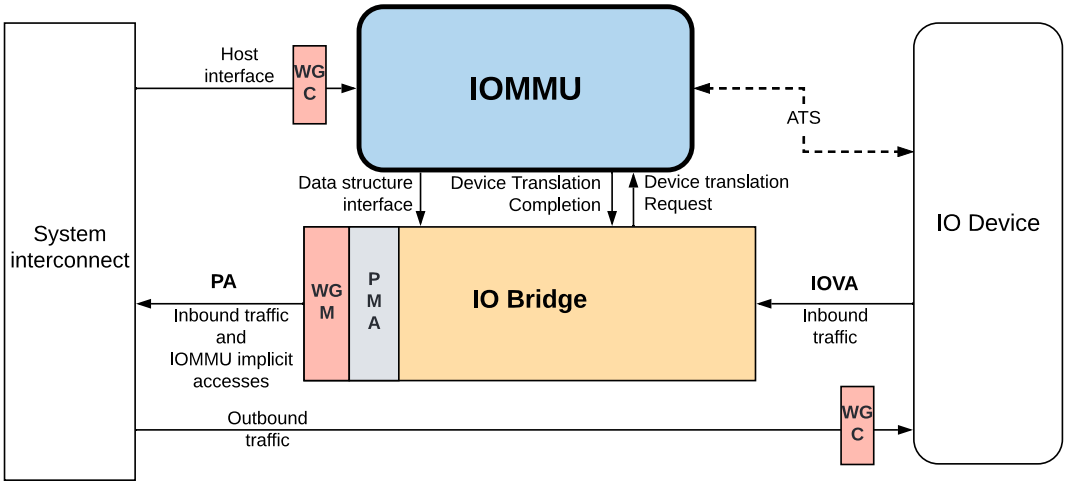


Figure 6: IOMMU Interfaces

2.1.1 Device Translation Request Interface

The Device Translation Request interface is an IOMMU slave interface, which receives the incoming virtual address requests from one or more IO devices. Along with the device address and transaction identifier, a HardContextID is provided to IOMMU in order to identify the source of the request and potentially its process. IOMMU does not receive the data of the device transactions.

2.1.2 Device Translation Completion Interface

The Device Translation Completion interface is an IOMMU master port that provides the requested translated Physical Address and an error status. As IOMMU may not respond to requests in order, this interface will also provide the transaction identifier as received on the

Device Translation Request interface. Optionally, IOMMU will also provide the permission rights of the translated transaction.

2.1.3 Host Interface

The Host interface is a slave interface to IOMMU for the harts to access IOMMU's internal registers and perform its global configuration or maintenance.

2.1.4 PDS Interface (IOMMU Data Structure Interface)

The PDS interface is an IOMMU master interface to the system interconnect to fetch the required data structure from system memory. This interface can be used by the Page Table Walker. This interface can be also used to fetch the context table entries and to access the various queues in system memory.

Note

The PDS port does not generate WRAP or FIXED bursts, and PDS AXI transactions do not cross a 64B boundary.

2.1.5 ATS Interface

The ATS interface is used to communicate with the PCIe Root Port. This interface is used to service an ATS request, to invalidate cached entries in the endpoint's ATC, and to request pages through the PCIe Page Request Interface mechanism. The AMBA DTI protocol for ATS messages is used to interface with the Root Port.

2.2 Software Interface

Software must use three different interfaces when using IOMMU:

1. A set of memory-mapped registers for IOMMU's configuration.
2. Memory-based structures:
    a. Context tables to map IO devices to Base Page Physical Number (PPN) of the appropriate page table
    b. Page Table Entries to map the Virtual Address to a Physical Address
3. Memory-based circular buffer queues:
    a. Command queue for context table configuration and for invalidation requests
    b. Fault queue for error logging from IOMMU
    c. PRI queue for PRI services

2.3 HardContextID Numbering

An incoming transaction on the device interface must include a request property, which will allow IOMMU to gather the proper Context for the address translation. This property is referred to as a HardContextID and must constructed with a DeviceID and a ProcessID. The information to construct the HardContextID {DeviceID, ProcessID} is carried on sideband signals. If more than one client device use IOMMU, the HardContextID is used to identify the source and potentially the process within each source.

The mapping of a physical device to the HardContextID is SoC IMPLEMENTATION DEFINED and must be known to system software. It is expected that an SoC would provide an explicit hardware logic to do this mapping or provide an implicit function to derive this mapping. DeviceID is of PARAMETRIZED size between 1 to 24 bits, (Parameter: `DIDWidth`). For IOMMU-22, the DeviceID width is limited to 16 bits. ProcessID is of PARAMETRIZED size between 0 to 20 bits, (Parameter: `PIDWidth`).

In IOMMU, the HardContextID is used to select the ContextID Table Entry (CTE) in the ContextID table. The maximum ContextID Table size is $2^{\texttt{HCIDWidth}}$.

2.3.1 HardContextID Numbering Examples

- 6-bit DeviceID and no ProcessID
  - Up to 64 ContextID table entries
  - In the case of DMA channels or privileged IO devices, each device has a unique DeviceID fixed by hardware.
  - In the case of a PCIe peripheral, if it is known that most-significant bits of the Bus or Device in the requesterID are not used, the HardContextID provided to IOMMU can then be reduced in size in order to limit the ContextID table size. For instance,

    ```
    HardContextID = DeviceID = {Bus[0]; Device[1:0]; Function[2:0]}
    ```

- 16-bit DeviceID and no ProcessID
  - Up to 64K ContextID table entries (assuming no PASID support)
  - Example of HardContextID derived from the PCIe RequesterID
  - `HardContextID = DeviceID = PCIe_RequesterID = {Bus[7:0]; Device[4:0]; Function[2:0]}`
- 36-bit HardContextID
  - Example of HardContextID derived from the PCIe RequesterID and PASID
  - `HardContextID = {DeviceID, ProcessID}`, where `DeviceID = PCIe_RequesterID[15:0]` and `ProcessID = PASID[19:0]`

- **NOTE:** If IOMMU is connected to several PCIe controllers, a full featured HardContextIID should be `{Seg_id, RequesterID, PASID}` where the Segment ID (`Seg_id`) would differentiate each PCIe controller.

The DeviceID and ProcessID are hardware identifiers which can be applied to PCIe systems as well as non-PCIe systems. They are used to determine translation rules (single stage and/or nested translation rules).

2.4 Data Structures and Translation Procedures

IOMMU uses internal data structures to get the translation rules relative to the inbound transactions. The data structure in IOMMU is the context table, which contains the translation table base pointer. This structure consists of a **Device Directory Table**, which provides context information on the DeviceID, and a **Process Directory Table**, which provides additional context information on the ProcessID of the DeviceID if a ProcessID is provided. When ProcessID is not supported, the context information is solely provided by the Device Directory.

The other data structure in system memory is the RISC-V page table, which is comprised of Page Table Entries (PTEs). These PTEs are also used by a hart's MMU to perform address translation.

A RISC-V core without the Hypervisor Extension is using its `satp` register to get the single-stage translation table base pointer (also referred to as Physical Page Number, or PPN) and access permission check is performed by the PMP (Physical Memory Protection used in RISC-V harts). The Device Directory and Process Directory tables contain similar information.

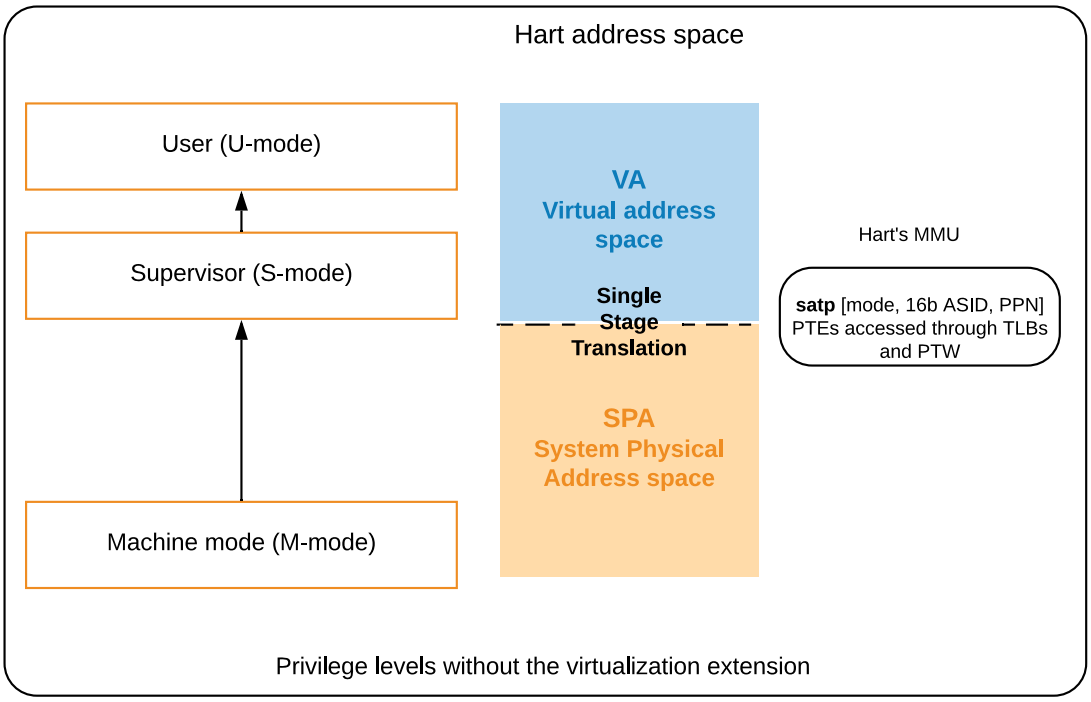


Figure 7: Privilege Levels Without the Virtualization Extension

A RISC-V core with the Hypervisor Extension provides two stages translation when the Virtualization mode is set. The `vsatp` register provides the VS-stage translation table base pointer and the `hgatp` register provides the G-stage translation table base pointer. The ContextID table entry also contains similar information.

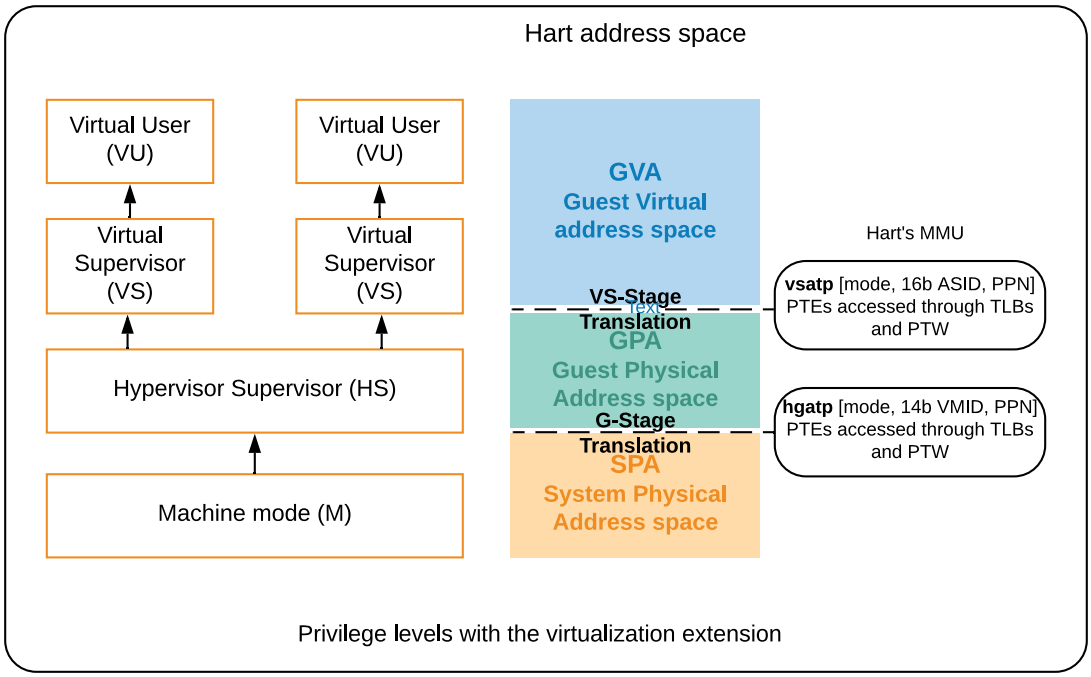


Figure 8: Privilege Levels With the Virtualization Extension

2.4.1 Context Tables

A context table entry describes each stage translation rule for a specific IO device. The DeviceID and ProcessID of an incoming transaction are used as an index in the context table to find the associated Base Page Physical Number (PPN) of the page table.

The context table actually consists of two tables: a Device Directory Table indexed by the DeviceID and a Process Directory Table indexed by the ProcessID. When no ProcessID is supported, the context entry is given solely by the Device Directory Table. The context for the translation is given by the Device Directory Table and the Process Directory Table when a ProcessID is provided. Both tables are stored in system memory.

The Device Directory table provides information on the device in order to perform address translation. The Process Directory table provides information on the address space used by the ProcessID on the specific DeviceID. The Device Directory table may be a multi-level table depending on the DeviceID width (determined by `DIDWidth`). The Device Directory Table holds the context information on the device.

The Process Directory table may be a multi-level table depending on the ProccessID width for the specific DeviceID. In the example below, the DeviceID width is 16 bits and the ProcessID for the DeviceID shown is 17 bits.

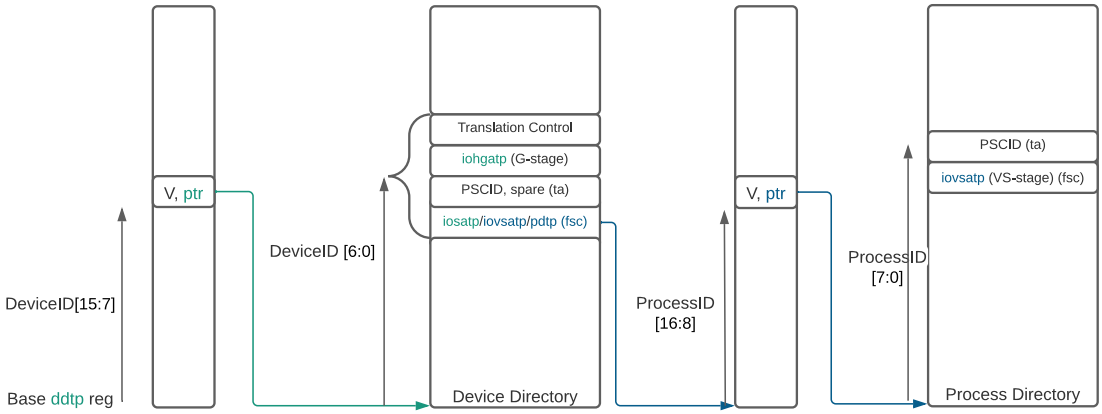


Figure 9: Multi-Level Context Table Example

The Device Directory can be single level if the DeviceID width is 7 bits or less, or it is a two level table if the DeviceID width is up to 16 bits, or it is a three level table if the DeviceID is up to 24 bits. The number of levels of the DeviceID Directory Table is determined by the `DIDWidth` parameter.

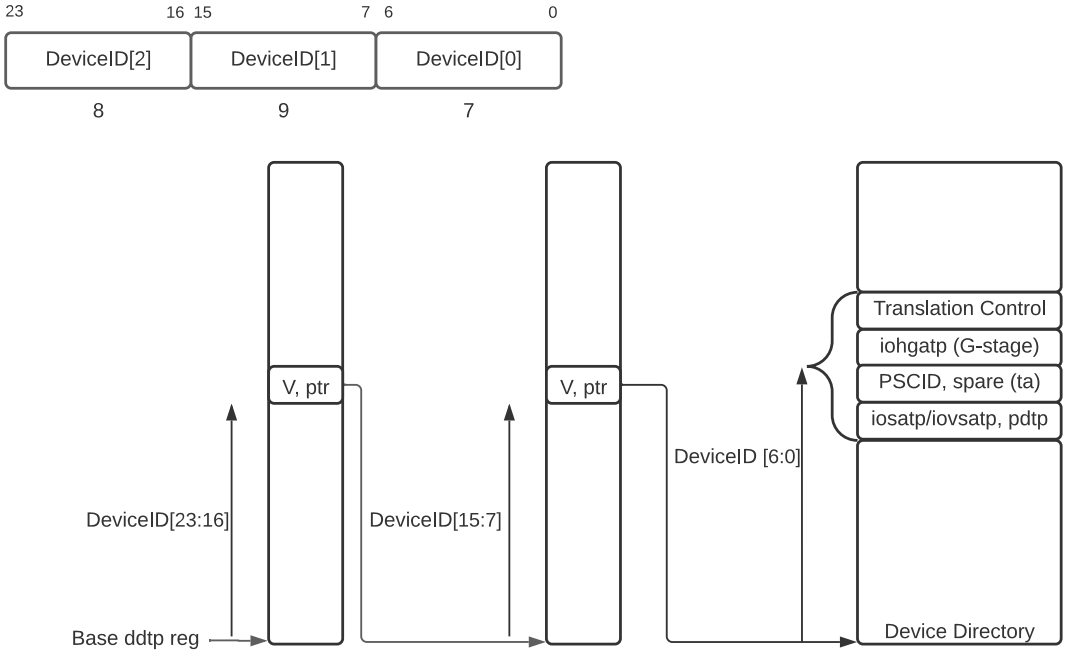


Figure 10: DeviceID Directory Table

The Process Directory Table may not exist if the DeviceID does not support ProcessID. If the ProcessID width for the DeviceID is 8 bits or less, the Process Directory is a single level table; if the ProcessID width is at most 17 bits, the Process Directory Table is a two-level table; and finally if the ProcessID width for the DeviceID is at most 20 bits, the Process Directory Table is a 3-level table.

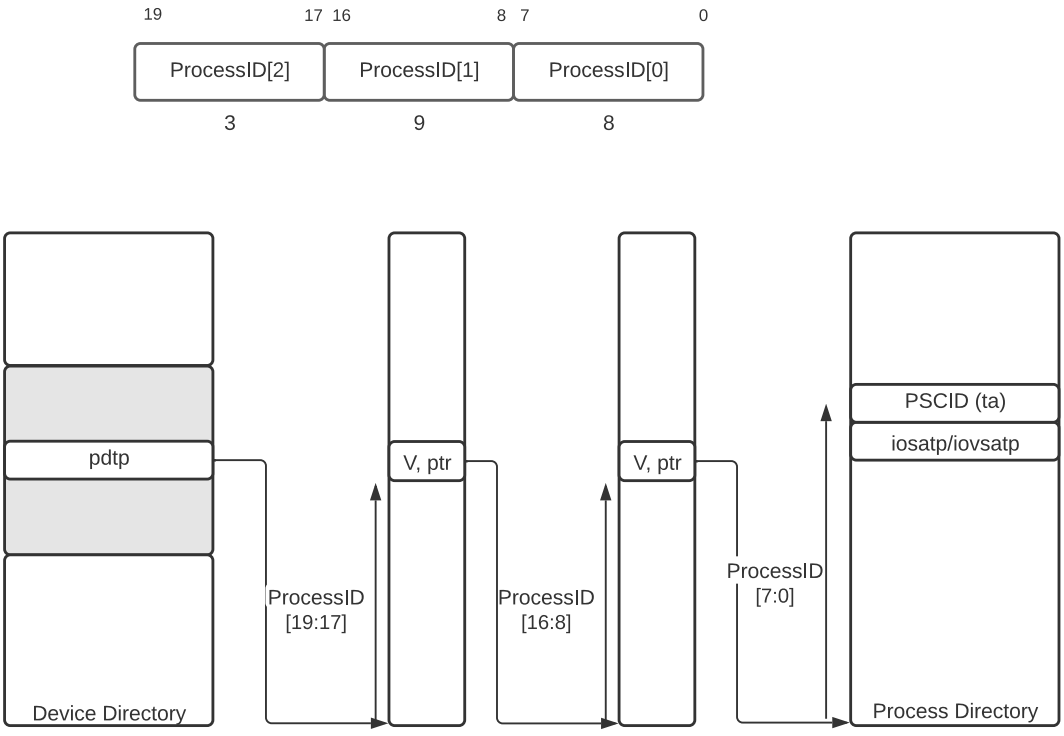


Figure 11: ProcessID Directory Table

For IOMMU to walk the DeviceID Directory Table or the ProcessID Directory Table, it reads each level of the table pointed by DeviceID[N] or ProcessID[N]. The non-leaf device directory entry or non-leaf process directory entry reads a descriptor as follows:

Bits	Field	Width	Description
0	V	1 bit	Valid. When set, it indicates that the descriptor is valid.
[9:1]	-	-	Reserved, set to 0.
[53:10]	PPN	44 bits	• Physical Page Number of the next level table (4KB page). • Physical address to the next level in the DeviceID Directory table. • Physical address or Guest Physical address to the next level in the ProcessID Directory table.
[63:54]	-	-	Reserved

Table 1: Non-Leaf Device or Process Directory Entry

Example: Assuming a 16b DeviceID, the DeviceID Directory table is a two level table. The `ddtp` register provides the base address (4KB aligned) of the first level of the DeviceID Directory. IOMMU reads the first level descriptor at `{ddtp.PPN, 12'b0} + {DeviceID[15:7], 3'b0}` and gets the next level PPN if the Valid bit is set. IOMMU reads the second and last level of the

DeviceID Directory table at `{descriptor.PPN, 12'b0} + {DeviceID[6:0], 5'b0}` to get the context information for this DeviceID.

2.4.2 Device Directory Entry

The following table describes the content of one Device Directory Table Entry (DDTE) which consists of 32B in system memory. The pointer to the leaf table entry must be 32B aligned.

Bits	DeviceID Directory Table Entry
[63:0]	Translation Control (`tc`)
[127:64]	IO Hypervisor Guest Address Translation and Protection (`iohgatp`)
[191:128]	Translation attributes (`ta`) - Process Software Context Identifier or Spare (`pscid`)
[255:192]	First Stage Control (`fsc`) - IO Supervisor / Virtual Supervisor Address Translation and Protection or Process Directory Table Pointer (`iosatp`/`iovsatp` or `pdtp`)

Table 2: Device Directory Table Entry (DDTE)

The Translation Control provides the following information:

Bits	Field	Width	Description
[63:10]	-	-	Reserved
9	DPE	1 bit	Default ProcessID Enable, when PDTV==1
[8:7]	-	-	Reserved
6	PRPR	1 bit	PRG-response-PASID-required. When set, it indicates that the IOMMU response message should include a PASID prefix if the associated "Page Request" had a PASID prefix else the IOMMU response message should not include a PASID prefix even if the associated "Page Request" had a PASID prefix.
5	PDTV	1 bit	Process Directory Table Valid. When set, it indicates that the DeviceID supports a ProcessID and that the Process Directory Table must be looked-up.
4	DTF	1 bit	Disable Translation Fault. When set, reporting of address translation faults for the device is disabled.
3	T2GPA	1 bit	Translate to GPA. When ATS is supported and enabled, the T2GPA when 1 causes IOMMU to return a GPA in response to translation requests. The permissions returned in translation requester are combined permissions and the page size resolved from the two stage translation. When T2GPA is 1 incoming translated requests carry a gpa and are translated through to a physical address.
2	EN_PRI	1 bit	PRI Enable. When set, the device is allowed to use PRI (EN_ATS is required).
1	EN_ATS	1 bit	ATS Enable. When set, the device is allowed to use ATS.
0	V	1 bit	Valid. When set, the ContextID Entry is valid.

Table 3: Translation Control (`tc`) Doubleword Format

The 32B device context is valid if `tv.V` is set. If it is 0, all other bits are don’t care.

Setting disable-translation-fault - `DTF` - bit to 1 disables reporting of faults encountered in the address translation process. Setting `DTF` to 1 does not disable error responses from being generated to the device in response to faulting transactions. Setting `DTF` to 1 does not disable reporting of faults from the IOMMU that are not related to the address translation process.

Note

A hypervisor may set `DTF` to 1 to disable fault reporting when it has identified conditions that may lead to a flurry of errors such as due to an abnormal termination of a virtual machine that may require the hypervisor to reset the device.

If IOMMU supports PCIe ATS specification (see capabilities register), EN_ATS bit is used to enable ATS transaction processing. If EN_ATS is set to 1, IOMMU supports the following inbound message transactions; otherwise they are treated as unsupported transactions.

- Translated read for execute transaction
- Translated read transaction
- Translated write/AMO transaction
- PCIe ATS Translation Request
- PCIe ATS Invalidation Completion Message
- PCIe ATS Page Request Message

If EN_ATS bit is 1 and the T2GPA bit is set to 1 the IOMMU returns a GPA the translation of an inbound VA in a TRANSLATION_REQUEST from the device. When T2GPA is 1, the inbound VA in translated memory accesses is a GPA and translated through the second-stage page table to a PA. This control enables a hypervisor to contain DMA from a device directly controlled by the guest OS, even with ATS capability enabled, to the VMs memory.

If EN_PRI bit is 0, then PCIe ATS Page Request messages from the device are invalid requests.

The IO Supervisor / Virtual Supervisor Address Translation and Protection or Process Directory Table Pointer field of the Device Context holds the context for first-stage translations. The field holds the pointer to a Process Directory Table if the PDTV bit is 1. If the PDTV bit is 0, this field instead holds a pointer to a Supervisor Address Translation and Protection register (i.e., iosatp) if iohgatp.MODE is BARE and holds a pointer to a Virtual Supervisor Address Translation and Protection (i.e., iovsatp) if iohgatp.MODE is not BARE.

The PDTV is expected to be set to 1 when the Device context is associated with a device that supports multiple process contexts and thus generates a valid ProcessID with its memory accesses.

When PDTV is 1, the DPE bit may be set to 1 to enable the use of 0 as the default value of process_id for translating requests without a valid process_id. In this case, the first stage translation rule is extracted from the Process Directory Table using ProcessID 0 .

When PDTV is 1 and DPE is set to 0, the first stage translation is treated as Bare.

When PDTV is 0, the DPE bit is reserved for future standard extension. Until its use is defined, the bit should be cleared by software for forward compatibility, and must be ignored by hardware.

Bits	Field	Width	Description
[63:60]	Mode	4 bits	Value for RISC-V paging scheme for G-stage translation: • 0: Bare, No translation nor protection • 8: Sv39x4, Page-based 41-bit virtual addressing • 9: Sv48x4, Page-based 50-bit virtual addressing • *10: Reserved, not supported in IOMMU-22* • *11: Reserved for future Sv64, Page-based 64-bit virtual addressing* • *Other values are reserved* When illegal values are programmed, a misconfigured error is raised.
[59:44	GSCID	16 bits	G-SoftContextID (similar to VMID) (ignored in Bare mode)
[43:0]	PPN	up to 44 bits	4KiB Physical Page Number for second G-stage translation (ignored in Bare mode)

Table 4: IO Hypervisor Guest Address Translation and Protection (`iohgatp`) Doubleword Format

- If `tc.pdtv` is set to 0, the DeviceID does not support a ProcessID.
  - If `iohgatp.Mode` is bare, the pdtp register holds the **iosatp** for IOMMU to perform the single stage translation.
  - If `iohgatp.Mode` is not bare, the pdtp register holds the **iovsatp** for IOMMU to perform the nested translation (VS-stage and G-stage translations)
  - The pscid holds the PSCID associated with the single stage (when `pdtp` register is iosatp) or VS-stage translation rule (when `pdtp` register is `iovsatp`) .
- If `tc.pdtv` is set to 1, the DeviceID supports a ProcessID. The width of the ProcessID is given by `pdtp.mode`. The Process Directory entry will provide the iosatp/iovsatp and pscid information for the single stage or VS-stage translation.
  - The Process Directory table holds the `iosatp` information when the `iohgatp.Mode` is bare.
  - The Process Directory table holds the `iovsatp` information when the `iohgatp.Mode` is not bare. In this case the pdtp pointer, the pointers in the Process Directory and the `iovsatp.PPN` are all Guest Physical Addresses (GPA) which must be translated using the iohgatp translation rule.

Bits	Field	Width	Description
[63:60]	Mode	4 bits	Value for RISC-V paging scheme for single stage translation: • 0: Bare, No translation nor protection • 8: Sv39, Page-based 39-bit virtual addressing • 9: Sv48, Page-based 48-bit virtual addressing • *10: Reserved, not supported in IOMMU-22* • *11: Reserved for future Sv64, Page-based 64-bit virtual addressing* • *Other values are reserved* When illegal values are programmed, a misconfigured error is raised.
[59:44]	-	-	Reserved
[43:0]	PPN	up to 44 bits	4KiB Physical Page Number for single stage translation

Table 5: IO Supervisor Address Translation and Protection (`iosatp`) Doubleword Format

Bits	Field	Width	Description
[63:60]	Mode	4 bits	Value for RISC-V paging scheme for VS-stage translation: • 0: Bare, No translation nor protection • 8: Sv39, Page-based 39-bit virtual addressing • 9: Sv48, Page-based 48-bit virtual addressing • *10: Reserved, not supported in IOMMU-22* • *11: Reserved for future Sv64, Page-based 64-bit virtual addressing* • *Other values are reserved* When illegal values are programmed, a misconfigured error is raised.
[59:44]	-	-	Reserved
[43:0]	PPN	up to 44 bits	4KiB **Guest** Physical Page Number for VS-stage translation

Table 6: IO Virtual Supervisor Address Translation and Protection (`iovsatp`) Doubleword Format

Bits	Field	Width	Description
[63:32]	-	-	Reserved
[31:12]	PSCID	up to 20b	Process SoftContextID
[11:0]	-	-	Reserved

Table 7: Process Software Context Identifier or Spare (`pscid`) Doubleword Format

The `PSCID` field provides the process soft-context ID that identifies the address-space of the process. `PSCID` facilitates address-translation fences on a per-address-space basis. The `PSCID` field in the device context is used as the address-space ID if `PDTV` is 0 and the `iosatp`/`iovsatp` `MODE` field is not Bare.

When `PDTV` is 1, the `pdtp` field holds the process-directory table pointer. When the device supports multiple process contexts, selected by the ProcessID, the PDT is used to determine the S-stage or VS-stage page table and associated `PSCID` for virtual address translation and protection.

The PDT is a 1, 2, or 3-level radix tree indexed using the process directory index (`PDI`) bits of the ProcessID. The pdtp field holds the PPN of the root page of the PDT and the `MODE` field that determines the number of levels of the PDT.

Bits	Field	Width	Description
[63:60]	Mode	4 bits	ProcessID Mode: • 0: Bare, No translation nor protection. • 1: PD08, 8-bit ProcessID enabled. The directory has 1 level. • 2: PD17, 17-bit ProcessID enabled. The directory has 2 levels. • 3: PD20, 20-bit ProcessID enabled. The directory has 3 levels. • *Other values are reserved* When illegal values are programmed, a misconfigured error is raised.
[59:44]	-	-	Reserved
[43:0]	PPN	up to 44 bits	PPN to the next level of Process Directory Table: 4KiB Guest Physical Page Number or Physical Page Number

Table 8: Process Directory Table Pointer (`pdtp`) Doubleword Format

IOMMU does not support the MSI single level page table and translates MSIs using the G-stage page table. Thus, the context information in the DeviceID Directory Table Entry consists of 32B.

2.4.3 Process Directory Entry

Bits	ProcessID Directory Table Entry
[63:0]	Translation Attributes (ta) - Process Software Context Identifier or Spare (pscid)
[127:64]	First Stage Control (fsc) - IO Supervisor / Virtual Supervisor Address Translation and Protection (iosatp/iovsatp)

Table 9: Process Directory Table Entry (PDTE)

Bits	Field	Width	Description
[63:32]	-	-	Reserved
[31:12]	PSCID	up to 20b	Process SoftContextID
[11:3]	-	-	Reserved
2	SUM	1 bit	When set, permits Supervisor User Memory access
1	ENS	1 bit	When set, ENables Supervisor access
0	V	1 bit	Valid. When set, the associated iosatp/iovsatp is valid

Table 10: PDT Translation Attributes (ta) Doubleword Format

The leaf PDT page is indexed by PDI[0] and holds the 16-byte process-context (PC).

A valid (V=1) leaf PDT entry holds the PPN of the root page of a Single stage or VS-stage page table and the MODE used to determine the paging scheme. The MODE field encodings are as defined for the MODE field in satp/vsatp CSR.

The software assigned process soft-context ID (PSCID) is used as the address space ID of the process identified by the Single stage or VS-stage page table.

When two-stage address translation is active (iohgatp.MODE != Bare), the PPN field holds a guest PPN of the VS-stage page table. When two-stage address translation is active, addresses of the VS-stage page table entries are then converted by guest physical address translation, as controlled by the iohgatp, into a supervisor physical address. A guest OS may thus directly edit the VS-stage page table to limit access by the device to a subset of its memory and specify permissions for the device accesses.

When enable-supervisory access (ENS) is 1, transactions requesting supervisor privilege are allowed with this ProcessID else the transaction is treated as an unsupported transaction.

If a DeviceID.ProcessID enables Supervisor access (ens), IOMMU must check the U bit on the VS-stage translation. This feature is supported only when the PASID prefix is present.

- When ENS == 0, transactions requesting supervisory privilege are disallowed for the specified DeviceID.ProcessID and thus unsupported. Requests without supervisor privilege (in the PASID TLP prefix) get a page fault if PTE.U == 0 on the first stage translation.

- When ENS == 1, transactions requesting supervisory privilege are allowed. The SUM (permit Supervisor User Memory access) bit modifies the privilege with which supervisor privilege transactions access virtual memory. When SUM=0, supervisor privilege transactions to pages mapped with U-bit in PTE set to 1 will fault.

When ENS is 1, supervisor privilege transactions that read with execute intent to pages mapped with U-bit in PTE set to 1 will fault, regardless of the state of SUM.

2.4.4 SoftContextID

Like the tuple {DeviceID, ProcessID} binds a hardware device function into a unique identifier, the SoftContextID is used by software to bind a process (Combination of VMID and ASID for instance) to a unique software identifier. This identifier consists of a tuple {GSCID, PSCID} and it is used to tag the base PPN (Physical Page Number) of a page table for address translation. The GSCID id is used to tag the G-stage TLB, the PSCID is used to tag the single stage TLB and the tuple {GSCID, PSCID} is used to tag the VS-stage TLB.

Software will keep track of the HardContextID ({DeviceID, ProcessID}) to SoftContextID ({GSCID, PSCID}) mapping.

The ContextID Table provides a maximum of 16-bit GSCID and 20-bit PSCID as it potentially maps nicely to a 14-bit VMID and 16-bit ASID or 20-bit PASID but it is a software implementation choice on how to handle the SoftContextID. However, software is not required to do a one-to-one mapping of a {VMID, ASID} tuple to a {GSCID, PSCID}.

2.4.5 Translation Operation

IOMMU gets an incoming request with a HardContextID sideband on its device translation request interface.

1. If IOMMU is globally disabled through `ddtp.iommu_mode` set to `Off`, the translation request is aborted, an error signal is raised on the completion interface and an Error Record is logged into the Fault Queue if enabled.
2. If IOMMU is already configured with `ddtp.iommu_mode` set to a level, the address translation is enabled. IOMMU can perform a context look up. Using the DeviceID as an index to the DeviceID Directory Table, it can read the Context Entry (CTE).

    a. An invalid CTE is found. An Error Record is logged and the error signal is raised on the completion interface.

    b. A valid CTE is found.

        - if a ProcessID is supported for this device (tc.pdtv is set), the ProcessID Directory Table must be walked.

3. The Translation mode is extracted from the CTE.

    - If bypass translation is requested, IOMMU will not perform any translation.

- If single stage translation is requested, the incoming address is a VA and the CTE contains a pointer to the base address (PPN) of the page translation table. The page table is walked using the iosatp.PPN and iosatp.Mode paging scheme.
- If G-stage only translation is requested, the incoming address is a GPA (Guest Physical Address) and the CTE contains a pointer to the base address (PPN) of the G-stage page translation table. The base address of the root page table must be 16KB aligned (lower two bits of the PPN in the CTE must be set to 0 otherwise a DC misconfigured fault). The page table is walked using the iohgatp.PPN and iohgatp.Mode paging scheme.
- If nested translation is requested, the incoming address is a GVA (Guest Virtual Address) and the iovsatp.PPN contains a pointer to the base address (PPN) of the VS-stage page translation table. This iovsatp.PPN address is actually a Guest Physical address which must be translated to a Supervisor Physical address thanks to the iohgatp.Mode paging scheme and ioghatp.PPN base address of the G-stage translation page table.

4. IOMMU accesses the PTEs in system memory pointed by context information to translate the request's address into a physical address. If an error occurs at any stage of PTE access, a fault is raised and logged as an Error Record.

   NOTE: Accessed and Dirty bits of PTEs are expected to be maintained by software and not modified in hardware by IOMMU.

5. IOMMU performs all required permission checks. The permission rules are provided by the PTE. In case of a violation, an Error Record is logged optionally.

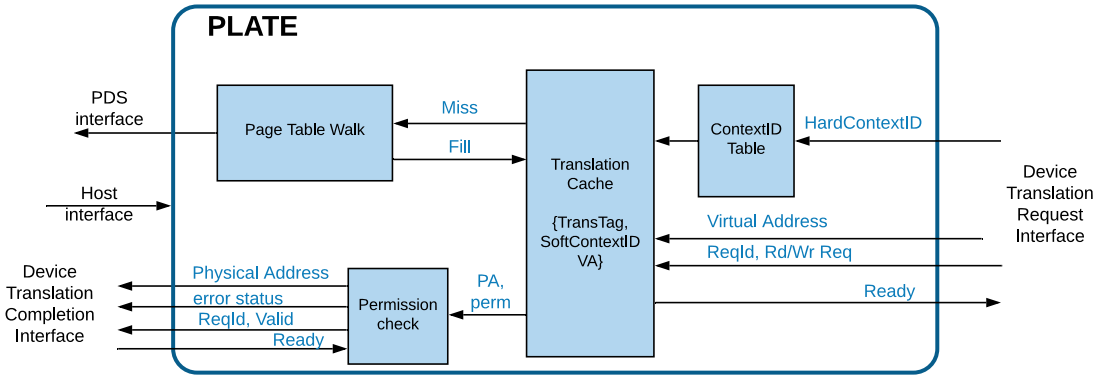


Figure 12: Inbound Request Flow through IOMMU

To avoid walking page tables on every inbound transaction, a translation cache will significantly improve IOMMU's performance. With the implementation of a TLB in IOMMU, the step 3 above in the translation operation becomes:

3. The CTE provides the translation mode, the PrivL and SoftContextID.

   a. In case of TLB hit: the TLB provides directly the PA and permission rules. Jump to step 5.

b. In case of TLB miss: the page table is walked: step 4.

Below is a summary of all possible translations through IOMMU:

`iohgatp.mode` (G-stage)	`iosatp.mode` or `iovsatp.mode`	Translation Mode	Address Translation	Paging Scheme
bare	bare	bypass	PA → PA	-
bare	!bare	Single Stage	VA → PA	`iosatp.mode`
!bare	bare	G-Stage only	GPA → SPA	`iohgatp.mode`

!bare	!bare	Nested	GVA → GPA
			GPA → SPA
				`iovsatp.mode`
				`iohgatp.mode`
				<5+a

Table 11: Summary of Translation Options

2.5 TLB Tagging

A Context table lookup returns the translation mode and all required information for IOMMU to perform the translation: the stages' PPN, the paging scheme to help the PTW, the SoftContextID and other information.

The Translation cache or TLB is tagged with a translation tag, the SoftContextID and the Virtual Address. The {GSCID, PSCID} is used to tag the VS-stage or Nested translation in the TLB, the GSCID is used to tag the G-stage translation and the PSCID is used to tag the single stage translation.

There may be several entries in the TLB with the same VA but different SoftContextID or even with the same VA and SoftContextID but different privilege level.

The translation tag (TransTag) consists of the privilege level and the translation mode as described in the following table.

TransTag	TMode	SoftContextID	Address Translation
SS	Single Stage	PSCID	VA → PA
VG	G-stage only	GSCID	GPA → SPA
VN	Nested translation	GSCID.PSCID	GVA → SPA

Table 12: Translation Tag (TransTag)

The TLB is not tagged with the {DeviceID,ProcessID} as multiple IO devices can use the same translation rules and be assigned to the same software context. However, a software context is uniquely identified by the privilege level and SoftContextID.

IOMMU's TLB entries can be invalidated using invalidation commands sent by software to the command queue. The queues are described in Section 2.9.1

Software can invalidate TLB entries based on the translation tag, SoftContextID and Virtual Address.

2.6 Address Size and Paging Scheme

RISC-V supports the following paging schemes:

Mode	Description
Sv39 in RV64	Page-based 39-bit virtual addressing
Sv39x4 in RV64	Page-based 41-bit virtual addressing (2-bit extension of Sv39)
Sv48 in RV64	Page-based 48-bit virtual addressing
Sv48x4 in RV64	Page-based 50-bit virtual addressing (2-bit extension of Sv48)
Sv57 in RV64	Page-based 57-bit virtual addressing
Sv57x4 in RV64	Page-based 59-bit virtual addressing (2-bit extension of Sv57)

Table 13: Paging Scheme Supported (SvN)

IOMMU may potentially work on various address sizes and paging schemes depending on the IO devices. On IOMMU's device interface, the address size is defined by a parameter (VAS: Device Virtual Address Size parameter).

On IOMMU's system interface or PDS interface, the address size is parametrized as it is fixed by the system physical memory capacity (PAS parameter: Physical Address Size parameter).

Even in bypass mode (when the inbound transaction goes through IOMMU without any address translation), IOMMU may raise an error if the incoming address is outside of the physical address width. An access violation may be raised.

In translation mode, if the incoming Virtual Address is outside the virtual addressing defined by the paging scheme for the translation, IOMMU may raise an error and send a page fault. If the computed translated address using the PTEs provides a PA outside of the PAS, IOMMU may raise an access violation.

In nested translation, if the result of the VS-stage provides a GPA wider than the virtual address size of the G-stage translation, a Guest page fault must be raised.

Finally, if the iosatp.PPN or iohgatp.PPN configured in a CTE are outside of the PAS, IOMMU may raise an access fault. In this case the address size check is performed on the PDS interface.

2.7 Page Table Walk

Page table entries used by IOMMU can be shared with the RISC-V cores. The Page Table Walker will comply with the RISC-V Instruction Set Manual Privilege Architecture. Depending on the paging scheme defined in the Context table, the PTW engine will go through the two-level

page table or three-level page table or four-level page table. For performance reasons, IOMMU may choose to implement several Page Table Walker in parallel and a Page Walker Cache.

The next figure shows the steps required for a single stage page walk in Sv48 for a 4KB page. In Sv48, the page table walker must go through a four-level page table. Each square represents a level of the page table.

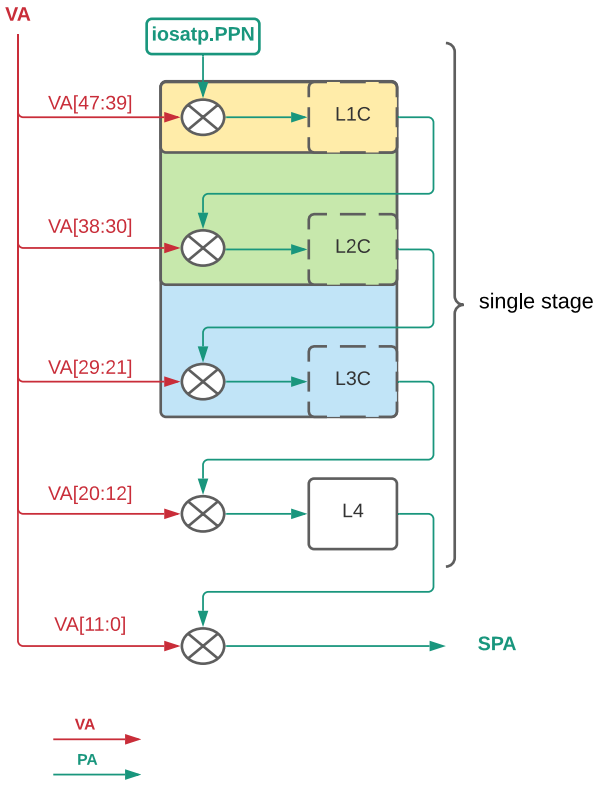


Figure 13: Single Stage Page Table Walk in Sv48

The next figure shows the steps required for a two dimension page walk for a 4KB page. The circles represent the level page table for the G-level stage translation and the squares represent the level page table for the VS-stage translation.

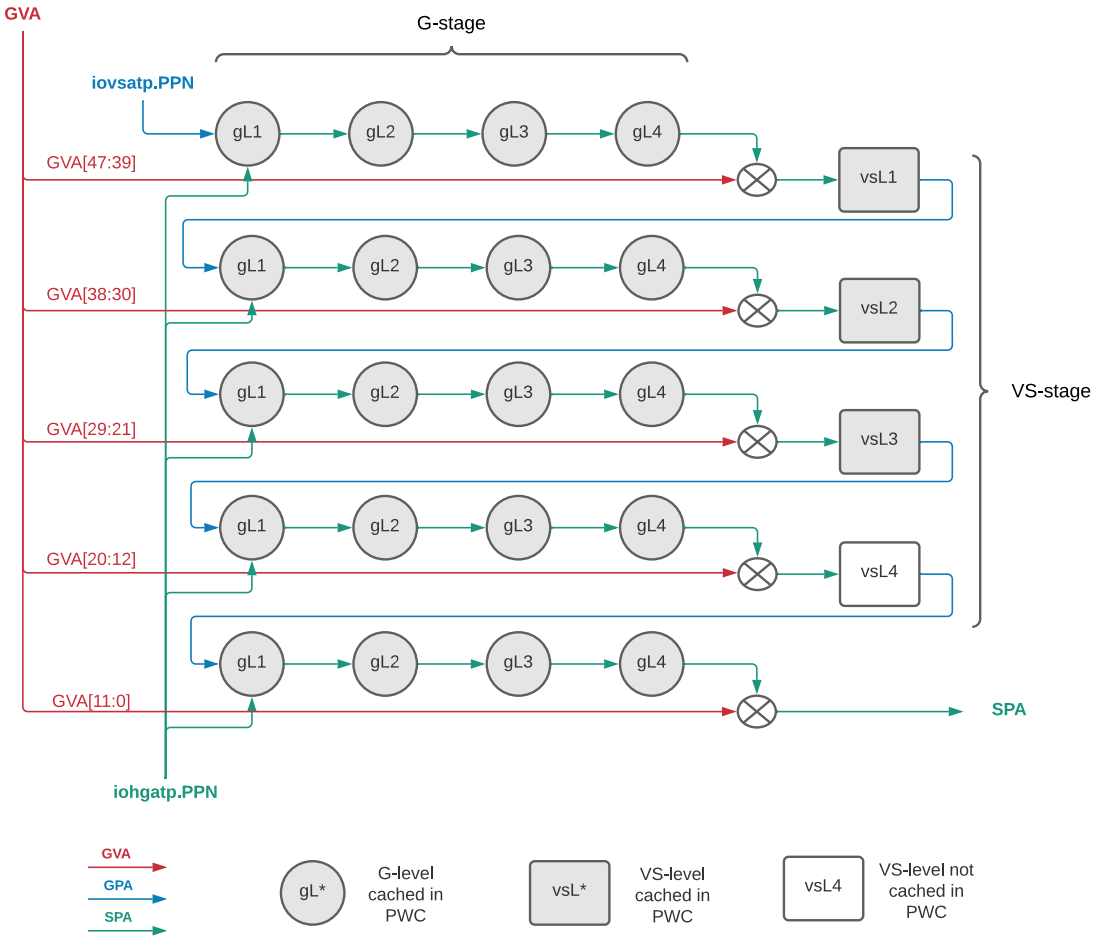


Figure 14: Nested Stage Page Table Walk in Sv48

Note

IOMMU does not update the A or D bits in hardware, but causes page faults or guest page faults based on the A and/or D bits being 0.

2.8 Virtualization

IOMMU translates IO device requests thereby protecting system software's memory space. When the IO device is managed by the Hypervisor Software using para-virtualization, the Guest device driver can use hypervisor calls so that the Hypervisor software can manage the DMA in the IO device. This software can arbitrate and schedule requests from the multiple Guest OS it manages.

IOMMU supports a full hardware virtualized system. With hardware-virtualized IOMMU, an IO device function can be mapped directly to the Guest OS which can then use native device drivers. The integration of IOMMU in an SoC provides efficient IO Virtualization. It allows the

hypervisor software to assign a physical device to a specific guest without danger of memory corruption to other virtual machines. IOMMU enforces the hypervisor policy on memory and system resource isolation for each guest virtual machine.

2.9 IOMMU's Queues

The host software interfaces with IOMMU thanks to queues implemented as circular buffers.

IOMMU implements three types of queues: the Command queue, the Fault queue, and optionally the Page-request queue. Each type of queue serves specific purposes:

- Command queue is used to request actions from IOMMU. It can invalidate one or several context entries, invalidate some translation entries stored in IOMMU's translation cache or send messages to the Root Port via the ATS interface.
- Fault queue is used by IOMMU to report some errors to the host software. The Fault Record can report some potential software or hardware errors such as page fault or illegal commands.
- Page-request queue (PQ) is used by IOMMU to report "Page Request" messages received from PCIe devices connected to the IOMMU. This queue is supported if the IOMMU supports PCIe ATS specification.

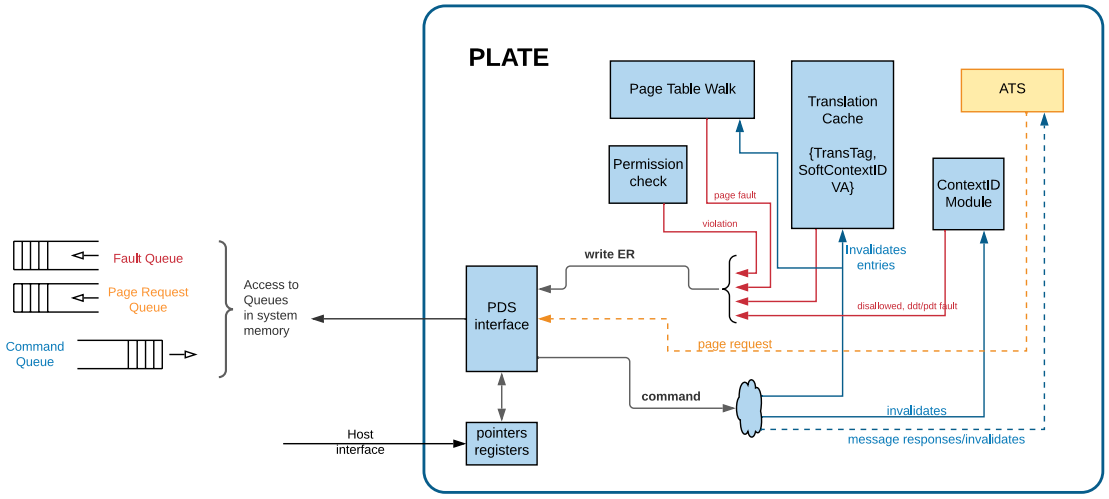


Figure 15: IOMMU's Queues

More details on the queues are provided in the next sections.

2.9.1 Command Queue

The host software is able to send commands to IOMMU via a circular buffer and via a couple of registers accessed through IOMMU's host interface. IOMMU holds both the head and tail pointers. The head pointer is maintained by IOMMU's hardware and it represents the first valid com-

mand of the queue. The tail pointer, once initialized, is managed by the host software and it points to the first empty slot in the queue.

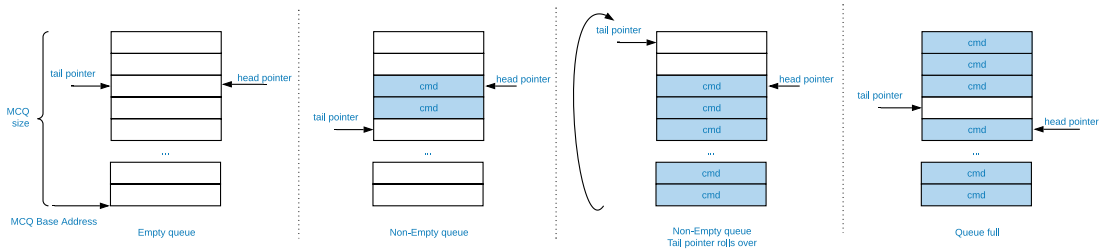


Figure 16: Circular Buffer Command Queue Example

The PPN of the base of this in-memory queue and the size of the queue is configured into a memory-mapped register called command-queue base (`cqb`) by software. The tail of the command-queue resides in a software controlled read/write memory-mapped register called `cqt`. The `cqt` is an index into the next command queue entry that software will write. Software first ensures that there are enough available slots in the queue before writing one or several commands in the queue. Subsequent to writing the command(s), software advances the `cqt` by the count of the number of commands written. The head of the command-queue resides in a read-only memory-mapped IOMMU controlled register called `cqh`. The `cqh` is an index into the command that IOMMU should process next. Subsequent to processing each command IOMMU advances the `cqh` by 1. If both the head and tail indexes are equal, then the command-queue is empty. If `cqt == (cqh - 1)` the command-queue is full.

If software sets an out of range tail pointer, the hardware behavior is unpredictable.

Software can send many commands in batches to IOMMU via the command queue and IOMMU will execute the commands at its own pace.

IOMMU reads commands from the queue in order but may execute the commands concurrently. If ordering is required, software must issue a `IOFENCE` command.

If IOMMU detects an illegal command, the command queue is stalled. The head pointer is not updated and points to the illegal command.

2.9.2 Fault Queue

Error reporting through IOMMU can be of two sorts:

- Transaction Errors from on the following interface:
  - Host interface
  - System memory interface
  - PDS interface
  - Device interface

- Internal IOMMU Errors:
  - Configuration errors
  - ContextID Table faults
  - TLB faults
  - PTW faults

All kinds of errors are reported by IOMMU through the following method:

- An Error Record is logged into the Fault queue, if these queues have been initialized and if the error is subject to logging.

Similar to the Command queue, the Fault queue is a circular buffer. IOMMU will write Error Records into the queue and the host software will read from the queue whenever the queue is not empty. IOMMU maintains the tail pointer and the host software maintains the head pointer.

Out of reset, the error logging mechanism is disabled. To enable this feature, software must:

- ensure that the Fault queue Status register is cleared,
- write the Head pointer such that the Head and Tail pointers are equal,
- set the Fault queue Enable bit in IOMMU's Fault queue Control register (`fqcsr`).

As soon as the Fault queue is full, IOMMU raises an overflow signal as it is unable to write a new Error Record. The logging of new errors is disabled until software empties the queue and clears the overflow bit in the status register.

2.9.3 Page Request Queue

The Page Request Queue is present if IOMMU's capability register indicates that it supports Address Translation Services per the PCIe standard. This queue is similar to the Fault Queue. The IOMMU is the provider and the hart or software is the consumer. This queue is used to transmit Page Request Messages from the Device to software in order to make a page present. Software responds to the device by sending a command to the Command queue.

2.10 Initialization

IOMMU initialization sequence:

1. Set up the interrupt capability
   - If MSI generation is supported, `msi_addr_*` , `msi_data_*` and `msi_vec_ctrl_*` must be set along with `icvec`
   - If wire interrupt is supported, nothing specific is required as the registers above are in this case hardcoded to 0.

2. Set up the Fault queue base address and size (`fqb`) then enable the queue and associated interrupt if desired in `fqcsr`.
3. Set up the Command queue base address and size (`cqb`) then enable the queue and associated interrupt if desired in `cqcsr`.
4. Set up the Page-request queue base address and size (`pqb`) then enable the queue and associated interrupt if desired in `pqcsr`.
5. Set up the hardware performance monitor if desired.
6. Set the Context table in system memory (Device Directory and Process Directory if supported)
7. Set the translation mode and the base address of the device directory in the `ddtp` register.
    - if 7-bit DeviceID or less is supported, set `ddtp.iommu_mode` to `1LVL`.
    - if 8-bit to 16-bit DeviceID is supported, set `ddtp.iommu_mode` to `2LVL`.

Chapter 3

Commands

The generic command format is a 16B command. IOMMU commands are grouped into a major command group determined by the `opcode` and within each group the `funct3` field specifies the function invoked by that command. The `opcode` defines the format of the operand fields. One or more of those fields may be used by the specific function invoked.

Bits	Field Name
[127:10]	operands
[9:7]	funct3
[6:0]	opcode

Table 14: Command Format

The encoding of the Command Code is as follows:

OPCODE	FUNCT3	Operands	Description
0x0	*0x0*		*Illegal Command*
IOTINVAL (0x1)	IOTINVAL.VMA (0x0)	PSCV, AV, PSCID, ADDR, GV, GSCID	Single stge or VS-stage translations invalidation
IOTINVAL (0x1)	IOTINVAL.GVMA (0x1)	PSCV, AV, PSCID, ADDR, GV, GSCID	G-stage translations invalidation
IOFENCE (0x2)	IOFENCE.C (0x0)	PR, PW, AV,WIS, ADDR, DATA	Command queue fence
IODIR (0x3)	IODIR.INVAL_DDT (0x0)	DV, DID	Device Directory cache invalidation
IODIR (0x3)	IODIR.INVAL_PDT (0x1)	DID, PID	Process Directory cache invalidation
ATS (0x4)	ATS.INVAL (0x0)	RID, PID, PV, DSEG, DSV, PAYLOAD	ATS Invalidation Request
ATS (0x4)	ATS.PRGR (0x1)	RID, PID, PV, DSEG, DSV, PAYLOAD	ATS Page Request Group Response

Table 15: Command Code List

Details on each command is provided in the next sections.

3.1 Invalidate Translations entries (IOTINVAL)

The IOMMU translation-table cache invalidation commands, IOTINVAL.VMA and IOTINVAL.GVMA synchronize updates to in-memory single stage, VS-stage and G-stage page table data structures with the operation of the IOMMU and flush the matching cached entries in IOMMU.

		Bits 127:126	Bits 125:74	Bits 73:64	Bits 63:60	Bits 59:44	Bits 43:34	Bits 33	Bits 32	Bits 31:12	Bit 11	Bit 10	Bits 9:7	Bits 6:0
		RSVD	ADDR[63:12]	RSVD	RSVD	GSCID	RSVD	GV	PSCV	PSCID	RSVD	AV	FUNCT3	OPCODE
Single stage and VS-stage translations invalidation	All single stage translations in host	0	0	0	0	0	0	0	0	0	0	0	IOTINVAL.VMA (0x0)	IOTINVAL (0x1)
	All single stage translations of a process in host - non-globals only	0	0	0	0	0	0	0	1	PSCID	0	0		
	Single address across all single stagepage tables in host - leaf entries, global and non-global	0	ADDR[63:12]	0	0	0	0	0	0	0	0	1		
	Single address of a process in host - leaf entries, non-global	0	ADDR[63:12]	0	0	0	0	0	1	PSCID	0	1		
	All VS-stage translations in a VM	0	0	0	0	GSCID	0	1	0	0	0	0		
	All VS-stage translations of a process in a VM - non-globals only	0	0	0	0	GSCID	0	1	1	PSCID	0	0		
	Single address across all VS-stage page tables of a VM - leaf entries, global and non-global	0	ADDR[63:12]	0	0	GSCID	0	1	0	0	0	1		
	Single address of a process in a VM - leaf entries, non-global	0	ADDR[63:12]	0	0	GSCID	0	1	1	PSCID	0	1		
		Bits 127:126	Bits 125:74	Bits 73:64	Bits 63:60	Bits 59:44	Bits 43:34	Bits 33	Bits 32	Bits 31:12	Bit 11	Bit 10	Bits 9:7	Bits 6:0
		RSVD	ADDR[63:12]	RSVD	RSVD	GSCID	RSVD	GV	RSVD	RSVD	RSVD	AV	FUNCT3	OPCODE
Second Stage translations invalidation	All G-stage page tables across all VMs (e.g. PMP change)	0	0	0	0	0	0	0	0	0	0	0	IOTINVAL.GVMA (0x1)	IOTINVAL (0x1)
	G-stage page table of a VM	0	0	0	0	GSCID	0	1	0	0	0	0		
	Single address in second stage page table of a VM - leaf entry	0	ADDR[63:12]	0	0	GSCID	0	1	0	0	0	1		

Figure 17: IOTINVAL Command

The GV operand indicates if the guest Soft-context ID (GSCID) operand is valid. The PSCV operand indicates if the process soft-context ID (PSCID) operand is valid. Setting PSCV to 1 is valid only with IOTINVAL.VMA. The AV operand indicates if the address (ADDR) operand is valid. When GV is 0, the translations associated with the host i.e., those where the second-stage translation is not active are operated on.

IOTINVAL.VMA guarantees that previous stores made to the single stage or VS-stage page tables by the harts are observed before all subsequent implicit reads from IOMMU to the corresponding page tables.

GV	AV	PSCV	Operation
0	0	0	invalidates cached information from any level of the single stage page table, for all host address spaces.
0	0	1	invalidates cached information from any level of the single stage page tables, but only for the host address space identified by PSCID operand. Accesses to global mappings are not ordered.
0	1	0	invalidates cached information from leaf single stage page table entries corresponding to the IOVA in ADDR, for all host address spaces.
0	1	1	invalidates cached information from leaf single stage page table entries corresponding to the IOVA in ADDR, for the host address space identified by PSCID operand. Global mappings may not be invalidated.
1	0	0	invalidates cached information from any level of the VS-stage page table, for all VM address spaces associated with GSCID.
1	0	1	invalidates cached information from any level of the VS-stage page tables, but only for the VM address space identified by PSCID and GSCID operand. Accesses to global mappings are not ordered.
1	1	0	invalidates cached information from leaf VS-stage page table entries corresponding to the IOVA in ADDR, for all VM address spaces associated with the GSCID operand.
1	1	1	invalidates cached information from leaf VS-stage page table entries corresponding to the IOVA in ADDR, for the VM address space identified by PSCID and GSCID operand. Global mappings may not be invalidated.

Table 16: Table IOTINVAL.VMA Operands and Operations

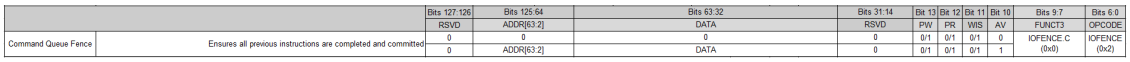
IOTINVAL.GVMA guarantees that previous stores made to the G-stage page tables are observed before all subsequent implicit reads from IOMMU to the corresponding G-stage page tables. Setting PSCV to 1 with IOTINVAL.GVMA is illegal.

AV	GV	Operation
-	0	Invalidates cached information from any level of the G-stage page table, for all VM address spaces.
0	1	Invalidates cached information from any level of the G-stage page tables, but only for all VM address spaces identified by the GSCID operand.
1	1	Invalidates cached information from leaf G-stage page table entries corresponding to the guest-physical-address in ADDR operand, for the all VM address spaces identified GSCID operand.

Table 17: Table IOTINVAL.GVMA Operands and Operations

3.2 Fence Command (IOFENCE)

The IOFENCE.C command provides a fence mechanism in IOMMU to ensure that all previous commands in this queue have completed. All older translation cache invalidations or CTE configurations are guaranteed to be done before IOMMU can read and execute any new commands from the queue. When the IOFENCE.C command completes, IOMMU can optionally signal its completion via an interrupt (wired or MSI)
Once the IOFENCE.C command is completed, IOMMU can work on the next available command from the queue.



		Bits 127:126	Bits 125:64	Bits 63:32	Bits 31:14	Bit 13	Bit 12	Bit 11	Bit 10	Bits 9:7	Bits 6:0
		RSVD	ADDR[63:2]	DATA	RSVD	PW	PR	WIS	AV	FUNCT3	OPCODE
Command Queue Fence	Ensures all previous instructions are completed and committed	0	0	0	0	0/1	0/1	0/1	0	IOFENCE.C (0x0)	IOFENCE (0x2)
		0	ADDR[63:2]	DATA	0	0/1	0/1	0/1	1		

Figure 18: Fence Command

The IOMMU fetches commands from the CQ in order but the IOMMU may execute the fetched commands out of order. The IOMMU advancing cqh is not a guarantee that the commands fetched by the IOMMU have been executed or committed.

A IOFENCE.C command guarantees that all previous commands fetched from the CQ have been completed and committed.

The commands may be used to order memory accesses from I/O devices connected to the IOMMU as viewed by the IOMMU, other RISC-V harts, and external devices or coprocessors. The PR and PW bits can be used to request that the IOMMU ensure that all previous requests from devices that have already been processed by the IOMMU be committed to a global ordering point such that they can be observed by all RISC-V harts and IOMMUs in the machine.

The wired-interrupt-signaling (WIS) bit when set to 1 causes a wired-interrupt from the command queue to be generated on completion of IOFENCE.C. This bit is reserved if the IOMMU supports MSI.

Note

Software should ensure that all previous read and writes processed by the IOMMU have been committed to a global ordering point before reclaiming memory made accessible to a device. A safe sequence for such memory reclamation is to first update the page tables to disallow access to the memory from the device and then use the IOTINVAL.VMA or IOTINVAL.GVMA appropriately to synchronize the IOMMU with the update to the page table. As part of the synchronization if the memory reclaimed was previously made read accessible to the device then request ordering of all previous reads; else if the memory reclaimed was previously made write accessible to the device then request ordering of all previous reads and writes to the IOFENCE. Ordering previous reads may be required if the reclaimed memory will be used to hold data that must not be made visible to the device.

The ordering guarantees are made for accesses to main-memory. For accesses to I/O memory, the ordering guarantees are implementation and I/O protocol defined.

Simpler implementations may unconditionally order all previous memory accesses globally.

The AV command operand indicates if ADDR operand and DATA operands are valid. If AV=1, the IOMMU writes DATA to memory at a 4-byte aligned address in ADDR operand as a 4-byte store.

Note

Software may configure the ADDR command operand to specify the address of the sereipnum_le/seteipnum_be in an IMSIC to cause an external interrupt notification on IOFENCE.C completion. Alternatively, software may program ADDR to a memory location and use IOFENCE.C to set a flag in memory indicating command completion.

3.3 Directory Cache Invalidation (IODIR)

The IODIR.INVAL_DDT command invalidates the Device Directory entry for the specified DeviceID if DV is set.

		Bits 127:64	Bits 63:40	Bits 39:34	Bits 33	Bits 32	Bits 31:12	Bit 11	Bit 10	Bits 9:7	Bits 6:0
		RSVD	DID	RSVD	DV	RSVD	RSVD	RSVD	RSVD	FUNCT3	OPCODE
Device directory cache invalidations	All device directory entries and process directory entries	0	0	0	0	0	0	0	0	IODIR.INVAL_DDT (0x0)	IODIR (0x3)
	Single device directory table entry and associated process directory entries	0	DID	0	1	0	0	0	0		
		Bits 127:126	Bits 125:64	Bits 63:40	Bits 39:34	Bits 33	Bits 32	Bits 31:12	Bit 11	Bit 10	Bits 9:7	Bits 6:0
		RSVD	DID		RSVD	RSVD	RSVD	PID	RSVD	RSVD	FUNCT3	OPCODE
Process directory cache invalidations	Specific process directory entry associated with a device	0	DID		0	0	0	PID	0	0	IODIR.INVAL_PDT (0x1)	IODIR (0x3)

Figure 19: IODIR.INVAL_DDT

When DV is set, the DeviceID specified in the command is valid.

This command invalidates the entries cached in IOMMU. IOMMU will need to fetch the new CTE from system memory when required. This command may be used if any field of the CTE has been modified in the system memory by software.

If DV is set to 0, this command invalidates all entries in both DeviceID Directory and ProcessID Directory. If DV is set to 1, this command invalidates the entry in the DeviceID Directory cache and all entries associated with the DeviceID in the ProcessID Directory cache.

DV	Behavior
0	This command invalidates all entries in both DeviceID Directory and ProcessID Directory cached in IOMMU.
1	This command invalidates the leaf entry associated with the specified DeviceID cached in IOMMU. If the DeviceID's context supports ProcessID, all ProcessID Directory entries associated with the DeviceID are invalidated.

Table 18: Directory Invalidation

The `IODIR.INVAL_PDT` command invalidates the leaf entry of the Process Directory for the specified DeviceID and ProcessID.

3.3.1 ATS Commands (ATS)

		Bits 127:64	Bits 63:56	Bits 55:40	Bits 39:34	Bits 33	Bits 32	Bits 31:12	Bit 11	Bit 10	Bits 9:7	Bits 6:0
		PAYLOAD	DSEG	RID	RSVD	DSV	PV	PID	RSVD	RSVD	FUNCT3	OPCODE
ATS Invalidation	Send invalidation to RID - no PASID prefix - specified payload	PAYLOAD	0	RID	0	0	0	0	0	0	ATS.INVAL (0x0)	ATS(0x4)
	Send invalidation to RID with PASID - specified payload	PAYLOAD	0	RID	0	0	1	PID	0	0		
	Send invalidation to RID on specified segment - no PASID prefix - specified payload	PAYLOAD	DSEG	RID	0	1	0	0	0	0		
	Send invalidation to RID on specified segment with PASID - specified payload	PAYLOAD	DSEG	RID	0	1	1	PID	0	0		
ATS Page group Resp	Send prgr to RID - no PASID prefix - specified payload	PAYLOAD	0	RID	0	0	0	0	0	0	ATS.PRGR (0x1)	
	Send prgr to RID with PASID - specified payload	PAYLOAD	0	RID	0	0	1	PID	0	0		
	Send prgr to RID on specified segment - no PASID prefix - specified payload	PAYLOAD	DSEG	RID	0	1	0	0	0	0		
	Send prgr to RID on specified segment with PASID - specified payload	PAYLOAD	DSEG	RID	0	1	1	PID	0	0		

Figure 20: ATS Commands

The ATS.INVAL command instructs the IOMMU to send a "Invalidation Request" message to the PCIe device function identified by RID (Requester ID). An "Invalidation Request" message is used to clear a specific subset of the address range from the address translation cache in a device function. The ATS.INVAL command completes when an "Invalidation Completion" response message is received from the device or a protocol defined timeout occurs while waiting for a response. The IOMMU may advance the cqh and fetch more commands from CQ while a response is awaited.

Note

Software that needs to know if the invalidation operation completed on the device may use the IOMMU command-queue fence command (IOFENCE.C) to wait for the responses to all prior "Invalidation Request" messages. The IOFENCE.C is guaranteed to not complete before all previously fetched commands were executed and completed. A previously fetched ATS command to invalidate device ATC does not complete till either the request times out or a valid response is received from the device.

The ATS.PRGR command instructs the IOMMU to send a “Page Group Response” message to the PCIe device function identified by the RID. The “Page Group Response” message is used by system hardware and/or software to communicate with the device functions page-request interface to signal completion of a “Page Request”, or the catastrophic failure of the interface.

If the PV operand is set to 1, the message is generated with a PASID TLP prefix with the PASID field set to the PID operand.

The PAYLOAD operand of the command is used to form the message body.

If the DSV operand is 1, then a valid destination segment number is specified by the DSEG operand.

Chapter 4

Memory Map

IOMMU's registers and queues are all memory mapped. Each register space in the table below is mapped on a different physical 4KB page and the location of the different queues is defined by the configuration registers. The required MMIO space is 8KB.

Data Structure	Base Address	Size
IOMMU Control Register space	`IOMMU_BASE`	4KB
IOMMU Error Bank	`IOMMU_BASE + 0x1000`	4KB
Command Queue	`IOMMU_CQB.PPN`	`LOG2SZ-1` entries of 16B
Fault Queue	`IOMMU_FQB.PPN`	`LOG2SZ-1` entries of 32B
Page-Request Queue	`IOMMU_PQB.PPN`	`LOG2SZ-1` entries of 16B

Table 19: IOMMU Memory Map

Note

`IOMMU_BASE` is the physical base address of IOMMU's Host interface in the system.

Table 20 lists the IOMMU memory-mapped registers. The control registers are described in Chapter 6. The performance monitor registers are described in Chapter 10.

Offset	Name	Description
0x0000	IOMMU_CAPABILITY	IOMMU Capabilities Register
0x0008	IOMMU_FCTL	IOMMU Feature Control Register
0x000C	IOMMU_CUSTOM0	IOMMU Configuration 0 Register
0x0010	IOMMU_DDTP	IOMMU DeviceID Directory Table Pointer Register
0x0018	IOMMU_CQB	IOMMU Command Queue Base Register
0x0020	IOMMU_CQH	IOMMU Command Queue Head Register
0x0024	IOMMU_CQT	IOMMU Command Queue Tail Register
0x0028	IOMMU_FQB	IOMMU Fault Reporting Queue Base Register
0x0030	IOMMU_FQH	IOMMU Fault Reporting Queue Head Register
0x0034	IOMMU_FQT	IOMMU Fault Reporting Queue Tail Register
0x0038	IOMMU_PQB	IOMMU Page-Request Queue Base Register
0x0040	IOMMU_PQH	IOMMU Page-Request Queue Head Register
0x0044	IOMMU_PQT	IOMMU Page-Request Queue Tail Register
0x0048	IOMMU_CQCSR	IOMMU Command-Queue Control and Status Register
0x004C	IOMMU_FQCSR	IOMMU Fault Reporting Queue Control and Status Register
0x0050	IOMMU_PQCSR	IOMMU Page-Request Queue Control and Status Register
0x0054	IOMMU_IPSR	IOMMU Interrupt Pending Status Register
0x0058	IOCNTOVF	Perf monitor overflow status register
0x005C	IOCNTINH	Perf monitor counter inhibition bits
0x0060	IOHPMMCYCLES	Perf Monitor cycle counter register
0x0068	IOHPM_COUNTER_1	Programmable performance monitor event counter 0
0x0070	IOHPM_COUNTER_2	Programmable performance monitor event counter 1
0x0078	IOHPM_COUNTER_3	Programmable performance monitor event counter 2
0x0080	IOHPM_COUNTER_4	Programmable performance monitor event counter 3
0x0088	IOHPM_COUNTER_5	Programmable performance monitor event counter 4
0x0090	IOHPM_COUNTER_6	Programmable performance monitor event counter 5
0x0098	IOHPM_COUNTER_7	Programmable performance monitor event counter 6
0x00A0	IOHPM_COUNTER_8	Programmable performance monitor event counter 7
0x00A8	IOHPM_COUNTER_9	Programmable performance monitor event counter 8

Table 20: IOMMU Control Registers List

Offset	Name	Description
0x00B0	IOHPM_COUNTER_10	Programmable performance monitor event counter 9
0x00B8	IOHPM_COUNTER_11	Programmable performance monitor event counter 10
0x00C0	IOHPM_COUNTER_12	Programmable performance monitor event counter 11
0x00C8	IOHPM_COUNTER_13	Programmable performance monitor event counter 12
0x00D0	IOHPM_COUNTER_14	Programmable performance monitor event counter 13
0x00D8	IOHPM_COUNTER_15	Programmable performance monitor event counter 14
0x0160	MHPMEVENT1	M-Event selector for counter 1
0x0168	MHPMEVENT2	M-Event selector for counter 2
0x0170	MHPMEVENT3	M-Event selector for counter 3
0x0178	MHPMEVENT4	M-Event selector for counter 4
0x0180	MHPMEVENT5	M-Event selector for counter 5
0x0188	MHPMEVENT6	M-Event selector for counter 6
0x0190	MHPMEVENT7	M-Event selector for counter 7
0x0198	MHPMEVENT8	M-Event selector for counter 8
0x01A0	MHPMEVENT9	M-Event selector for counter 9
0x01A8	MHPMEVENT10	M-Event selector for counter 10
0x01B0	MHPMEVENT11	M-Event selector for counter 11
0x01B8	MHPMEVENT12	M-Event selector for counter 12
0x01C0	MHPMEVENT13	M-Event selector for counter 13
0x01C8	MHPMEVENT14	M-Event selector for counter 14
0x01D0	MHPMEVENT15	M-Event selector for counter 15
0x0258	TREQIOVA	Translation-request IOVA
0x0260	TRREQCTL	Translation-request control
0x0268	TRRESP	Translation-response
0x02B0	VERSIONID	IOMMU RTL Version Register
0x02B4	IOMMU_DTISR	IOMMU DTI-ATS Status register
0x02B8	IOMMU_TIMER	IOMMU ATS Timeout Countdown register
0x02F8	IOMMU_ICVEC	IOMMU Interrupt Cause Vector Register
0x0300	IOMMU_MSI_ADDRESS_0	IOMMU MSI ADDRESS Register 0
0x0308	IOMMU_MSI_DATA_0	IOMMU MSI DATA Register 0
0x030C	IOMMU_MSI_CTL_0	IOMMU MSI CTL Register 0
0x0310	IOMMU_MSI_ADDRESS_1	IOMMU MSI ADDRESS Register 1
0x0318	IOMMU_MSI_DATA_1	IOMMU MSI DATA Register 1
0x031C	IOMMU_MSI_CTL_1	IOMMU MSI CTL Register 1
0x0320	IOMMU_MSI_ADDRESS_2	IOMMU MSI ADDRESS Register 2
0x0328	IOMMU_MSI_DATA_2	IOMMU MSI DATA Register 2

Table 20: IOMMU Control Registers List

Offset	Name	Description
0x032C	IOMMU_MSI_CTL_2	IOMMU MSI CTL Register 2
0x0330	IOMMU_MSI_ADDRESS_3	IOMMU MSI ADDRESS Register 3
0x0338	IOMMU_MSI_DATA_3	IOMMU MSI DATA Register 3
0x033C	IOMMU_MSI_CTL_3	IOMMU MSI CTL Register 3
0x1000	SUMMARY_STATUS	Error Summary Status Register
0x1008	ERROR_REVISION_PERSONALITY	Error Bank Revision/Personality Register
0x1010	ERROR_STATUS	Error Entry Status Register
0x1018	ERROR_SUPPLEMENTAL_0	Error Bank Supplemental 0 Register
0x1028	CONFIG_SIMPLE_MODIFIER	Error Bank Simple Modifier Register

Table 20: IOMMU Control Registers List

Chapter 5

Programming Model

5.1 Register Definition

See Chapter 6 for a detailed description of IOMMU-22 registers.

When a single-wire interrupt is supported, then `msi_add_*` , `msi_data_*` , `msi_vec_ctrl_*` and `icvec` registers are hardcoded to 0.

5.2 Sequence to Set up the Command queue

Out of reset, `cqh` and `cqt` are both set to 0, `cqb` is also set to 0.
To enable the Command queue, software needs first to set `cqb` to define the queue base address in system memory and its size. Then software can set `cqcsr.cqen` to 1 in order to enable the queue. At this stage, the QM Manager will monitor the command queue fill level.

5.3 Sequence to invalidate one CID entry

1. Software must read the head pointer (`cqh`) to make sure there are enough available slots to store the new `IODIR` command.
2. If at least 1 slot is available, software can write the command at its tail.
3. Software updates the tail pointer in the IOMMU-22 `cqt` register.

5.4 Sequence to invalidate a translation rule

1. Software must read the head pointer (`cqh`) to make sure there are enough available slots to store the new `IOTINVAL` command.
2. If at least 1 slot is available, software can write the command at its tail.
3. Software updates the tail pointer in the IOMMU-22 `cqt` register.

5.5 Sequence to read an Error Record

1. Upon reception of the interrupt (wired INT interrupt or MSI via IMSIC), software must first read the pending status register `ipsr`. If the status bit (`fip`) is set, it indicates that the fault queue generated an interrupt. Software must then read the `fqcsr` register. If `fqof` and `fqmf` are clear, it means that at least one error occurred. The head and tail pointers are not equal.
2. Read the Error Record at the head of the queue.
3. Once the error is handled, software must update the head pointer in IOMMU-22 by setting `fqh`.
4. If the interrupt is still pending, software must read the next error record.

5.6 Sequence to re-enable an Error Record queue after an overflow

1. When IOMMU-22 is unable to store any new error records because the queue is full, the overflow status bit in the `fqcsr` is set. The hardware logging mechanism is stopped (which means that some Error Records may be silently dropped).
2. Software must empty the Error Record queue by reading all slots.
3. Software must set the `fqh` register to the tail pointer (read in `fqt`).
4. Finally, software must clear the Overflow bit in order re-enable the error logging in the queue (Write one to clear in `fqcsr`).

Chapter 6

Register Formats

6.1 IOMMU Capabilities Register

This register identifies the IOMMU IP block, its version and capabilities.

IOMMU Capabilities Register (`IOMMU_CAPABILITY`)				
Register Offset		`0x0`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[7:0]	`SPEC_VERSION`	RO	`0x10`	IOMMU Specification Version. The low nibble is used to hold the minor version of the specification and the upper nibble is used to hold the major version of the specification. For example, an implementation that supports version 1.0 of the specification reports `0x10`.
8	`SV32`	RO	`0x0`	Page-based 32-bit virtual addressing is supported
9	`SV39`	RO	`0x1`	Page-based 39-bit virtual addressing is supported
10	`SV48`	RO	`0x1`	Page-based 48-bit virtual addressing is supported
11	`SV57`	RO	`0x0`	Page-based 57-bit virtual addressing is supported
[14:12]	Reserved			
15	`SVPBMT`	RO	`0x1`	Page-based memory types is supported
16	`SV32X4`	RO	`0x0`	Page-based 34-bit virtual addressing for G-stage translation is supported
17	`SV39X4`	RO	`0x1`	Page-based 41-bit virtual addressing for G-stage translation is supported
18	`SV48X4`	RO	`0x1`	Page-based 50-bit virtual addressing for G-stage translation is supported
19	`SV57X4`	RO	`0x0`	Page-based 59-bit virtual addressing for G-stage translation is supported
[21:20]	Reserved			

Table 21: IOMMU Capabilities Register

22	MSI	RO	0x0	MSI address translation using Write-through mode MSI PTE is supported
23	MRIF	RO	0x0	MSI address translation using MRIF mode MSI PTE is supported
24	AMO	RO	0x0	IOMMU supports A/D bit updates to first and second stage page tables. If MRIF is supported, then AMO updates to MRIF are supported.
25	ATS	RO	0x1	ATS/PRI support
26	T2GPA	RO	0x1	Returning guest-physical-address (GPA) in ATS translation completion is supported
27	END	RO	0x0	Endianness support. When 0, IOMMU supports one endianness (either little or big). When 1, IOMMU supports both endianness. The endianness is defined in the IOMMU_FCTL register. **Note: IOMMU-22 only supports Little Endianness.**
[29:28]	IGS	RO	0x2	IOMMU interrupt generation support. • 0x0 - MSI: IOMMU supports only MSI generation • 0x1 - WIS: IOMMU supports only wired interrupt generation • 0x2 - BOTH: IOMMU supports both MSI and wired interrupt generation. The interrupt generation method must be defined in the IOMMU_FCTL register. • 0x3 - *Reserved for standard use*
30	PMON	RO	0x1	HW performance monitor is present
31	DEBUG	RO	0x1	Debug enable. Note that this field is volatile and may be masked in IP-XACT.
[37:32]	PAS	RO	0x30	Physical Address Size
38	PD8	RO	0x1	One level PDT with 8-bit process_id supported
39	PD17	RO	0x1	Two level PDT with 17-bit process_id supported
40	PD20	RO	0x1	Three level PDT with 20-bit process_id supported
[63:41]	Reserved			

Table 21: IOMMU Capabilities Register

Note

Hypervisor may provide a software-emulated IOMMU to allow the guest to manage the VS-stage page tables for fine grained control on memory accessed by guest controlled devices.

A hypervisor that provides such an emulated IOMMU to the guest may retain control of the G-stage page tables and clear the `SV*X4` fields of the emulated `IOMMU_CAPABILITY` register.

A hypervisor that provides such an emulated IOMMU to the guest may retain control of the MSI page tables used to direct MSI to guest interrupt files in an IMSIC or to a memory-resident-interrupt-file and clear the `MSI` and `MRIF` fields of the emulated `IOMMU_CAPABILITY` register.

IOMMU-22 does not support the following options:

- Sv32, Sv32x4, Sv57, Sv57x4
- `MSI_FLAT`, `MSI_MRIF`
- `AMO`, `AMO_MRIF`
- `END`

6.2 IOMMU Feature Control Register

If software enables or disables a feature when the IOMMU is not OFF (i.e., `IOMMU_DDTP.DDTP_IOMMU_MODE == Off`) then the IOMMU behavior is `UNSPECIFIED`.

IOMMU Feature Control Register (`IOMMU_FCTL`)				
Register Offset		`0x8`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
0	Reserved			
1	`WSI`	WARL	`0x0`	Wire-signaled interrupt enable. When 1, IOMMU interrupts are signaled as wired interrupts.
[7:2]	Reserved			

Table 22: IOMMU Feature Control Register

6.3 IOMMU Configuration 0 Register

The `IOMMU_CUSTOM0` register holds the following fields:

IOMMU Configuration 0 Register (IOMMU_CUSTOM0)				
Register Offset		0xC		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[7:0]	CFG_TIMEOUT	RW	0x2C	Set the countdown value for the timeout timer. When set to N, the countdown is initialized to $N*2^{30}$ clock cycles and when triggered by an IOFENCE command, it decrements at every clock cycle. A timeout interrupt is raised when the counter reaches 0 unless the command completed in a timely manner. Default value: assuming a 500MHz clock, CFG_TIMEOUT is set to 44 which corresponds to 90s. Assuming a 1GHz clock, CFG_TIMEOUT should be set to 90 which corresponds to 90s. Assuming a 2GHz clock, CFG_TIMEOUT should be set to 180 which corresponds to 90s. Recommendation: Set the countdown timer appropriately to timeout after 90s.
8	CFG_MPAGE	RW	0x0	When set, the TLB supports mega-page indexing
9	CFG_GPAGE	RW	0x0	When set, the TLB supports giga-page indexing
10	CFG_TPAGE	RW	0x0	When set, the TLB supports tera-page indexing
11	CFG_PPAGE	RW	0x0	When set, the TLB supports peta-page indexing
12	PTE_DISABLE	RW	0x0	When set, the cache in the Page Table Walk is disabled
13	CTE_DISABLE	RW	0x0	When set, the intermediate level cache in the Context Table Walk is disabled (only leaf cache is functional)
14	CG_DIS	RW	0x0	When set, clock gating is disabled within the IOMMU
15	Reserved			

Table 23: IOMMU Configuration 0 Register

Once IOMMU is in TRANSLATION mode after its initialization, any modification to the translation parameters in this register would result in unpredictable behavior. If any field in IOMMU_CUSTOM0 must be modified, software must first ensure that there are no in-flight or incom-

ing device translation requests. Then it must invalidate all IOMMU cached entries and finally make the necessary modifications.

6.4 IOMMU DeviceID Directory Table Pointer Register (RW)

The `IOMMU_DDTP` register holds the following fields:

IOMMU DeviceID Directory Table Pointer Register (`IOMMU_DDTP`)				
Register Offset		`0x10`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	`DDTP_IOMMU_MODE`	WARL	`0x0`	Device directory table mode. • `0x0` - `Off`: No inbound memory transactions are allowed by the IOMMU • `0x1` - `Bare`: No translation nor protection. All inbound memory accesses are passed through. • `0x2` - `1LVL`: The directory has 1 level - the device context is 64B if `IOMMU_CAPABILITY.MSI` is 1 else 32B • `0x3` - `2LVL`: The directory has 2 levels - the device context is 64B if `IOMMU_CAPABILITY.MSI` is 1 else 32B • `0x4` - `3LVL`: The directory has 3 levels - the device context is 64B if `IOMMU_CAPABILITY.MSI` is 1 else 32B
4	`DDTP_BUSY`	RO	`0x0`	DDTP is busy
[9:5]	Reserved			
[31:10]	`DDTP_PPN_LO`	WARL	`0x0`	Physical Page Number (low part) of the DeviceID Directory Table's root (page size is 4KB)
[45:32]	`DDTP_PPN_HI`	WARL	`0x0`	Physical Page Number (high part) of the DeviceID Directory Table's root (page size is 4KB)
[63:46]	Reserved			

Table 24: IOMMU DeviceID Directory Table Pointer Register

All IOMMU must support `IOMMU_DDTP.DDTP_IOMMU_MODE` "Off" and "Bare" mode. An IOMMU is allowed to support a subset of directory table modes and device contexts. At a minimum one of the modes must be supported.

When the MODE field value is changed the IOMMU guarantees that in-flight transactions from devices connected to the IOMMU will be processed with the configurations applicable to the old value of the MODE field and that all transactions and previous requests from devices that have already been processed by the IOMMU be committed to a global ordering point such that they can be observed by all RISC-V hart, devices, and IOMMUs in the machine

`IOMMU_DDTP.DDTP_BUSY`: A write to `IOMMU_DDTP` may require the IOMMU to perform many operations that may not occur synchronously to the write. When a write is observed by the `IOMMU_DDTP`, the busy bit is set to 1. When the busy bit is 1, behavior of additional writes to the `IOMMU_DDTP` is implementation defined. Some implementations may ignore the second write and others may perform the actions determined by the second write. Software must verify that the busy bit is 0 before writing to the `IOMMU_DDTP`. If the busy bit reads 0 then the IOMMU has completed the operations associated with the previous write to `IOMMU_DDTP`.

`IOMMU_DDTP.DDTP_PPN`: page frame number of root DDT table. When the PPN field value is changed, the IOMMU guarantees that in-flight transactions from devices connected to the IOMMU will be processed with the configurations applicable to the old value of the ppn field and that all transactions and previous requests from devices that have already been processed by the IOMMU be committed to a global ordering point such that they can be observed by all RISC-V hart, devices, and IOMMUs in the machine.

Note

`DDTP_PPN` is a WARL field because the IOMMU may not support 56b of physical address space. For instance, if the system supports 47 bits of physical address space, the lower 35 bits of the PPN are RW and the upper 9 bits are hardcoded to 0.

6.5 Command Queue Base Register

This 64-bit register holds the Command queue base PPN (44-bit) and its size. The queue base address and size register is programmable (RW) as this queue can be located in system memory.

IOMMU Command Queue Base Register (IOMMU_CQB)				
Register Offset		0x18		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	LOG2SZ-1	WARL	0x0	Number of commands in the command queue as a log to base 2 minus 1. A value of 0 indicates a queue of size 2 entries. If the Command Queue has 256 or fewer entries then the base address of the queue is always aligned to a 4-KiB page address. If the Command Queue has more than 256 entries then the CQ base address must be naturally aligned to 2LOG2SZ x 16.
[9:4]	Reserved			
[31:10]	PPN_LO	WARL	0x0	Physical page number (low part) pointing to the base of the Command queue
[45:32]	PPN_HI	WARL	0x0	Physical page number (high part) pointing to the base of the Command queue
[63:46]	Reserved			

Table 25: IOMMU Command Queue Base Register

Note

IOMMU may implement 4 bits of LOG2SZ-1. The upper bit is hardcoded to 0.

6.6 Command Queue Head Register

This 32-bit register holds the head pointer of the queue. This register is read only as it is updated by IOMMU's hardware and is provided to software for information only.

IOMMU Command Queue Head Register (IOMMU_CQH)				
Register Offset		0x20		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	INDEX	RO	0x0	Head Index (maintained by IOMMU)

Table 26: IOMMU Command Queue Head Register

6.7 Command Queue Tail Register

This 32-bit register holds the tail pointer of the queue. This register is a read/write register. It is updated by host software to let IOMMU's hardware know when one or several new entries in the queue have been updated.

IOMMU Command Queue Tail Register (IOMMU_CQT)				
Register Offset		0x24		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[15:0]	INDEX	WARL	0x0	Tail Index (only LOG2SZ bits are writeable)

Table 27: IOMMU Command Queue Tail Register

6.8 Command Queue Control and Status Register

This 32-bit RW register holds the following control bits:

IOMMU Command Queue Control and Status Register (IOMMU_CQCSR)				
Register Offset		0x48		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
0	CQEN	RW	0x0	Command Queue enable. SW sets to 1 to request CQ to be enabled. The CQ is enabled when CQON reads 1. SW sets to 0 to request CQ to be disabled. The CQ is disabled when CQON reads 0.
1	CIE	RW	0x0	Command queue interrupt-enable. When set to 1, enables generation of interrupts from CQ.
[7:2]	Reserved			
8	CQMF	RW1C	0x0	Command queue access lead to a memory access fault. The queue stalls until this bit is cleared. An interrupt is generated if not already pending (i.e., IOMMU_IPSR.CIP == 1). Note that this field is volatile and may be masked in IP-XACT.
9	CMD_TO	RW1C	0x0	If execution of an instruction leads to a timeout (e.g., a command to invalidate device ATC may timeout waiting for a completion), then the CQ sets the CMD_TO bit and stops processing from the CQ. When CMD_TO is set to 1, an interrupt is generated if not already pending (i.e., IOMMU_IPSR.CIP is 1)

Table 28: IOMMU Command Queue Control and Status Register

				and not masked by CIE. To re-enable command processing software should clear this bit by writing 1. Note that this field is volatile and may be masked in IP-XACT.
10	CMD_ILL	RW1C	0x0	If an illegal or unsupported command is fetched and decoded by the CQ then the CQ sets the CMD_ILL bit and stops execution from the CQ. When CMD_ILL is set to 1, an interrupt is generated if not already pending (i.e., IOMMU_IPSR.CIP is 1) and not masked by CIE. To re-enable command processing software should clear this bit by writing 1. Note that this field is volatile and may be masked in IP-XACT.
11	FENCE_W_IP	RW1C	0x0	Indicates a wired interrupt pending due to execution of an IOFENCE.C command. An interrupt is generated on setting FENCE_W_IP if not already pending (i.e., IOMMU_IPSR.CIP == 1) and FENCE_W_IP is 0. SW clears by writing 1. This interrupt can be raised even if IOMMU_CQCSR.CIE == 0 as it has been requested by a SW command. Note that this field is volatile and may be masked in IP-XACT.
[15:12]	Reserved			
16	CQON	RO	0x0	The command-queue enable (CQEN) bit enables the command-queue when set to 1. Changing CQEN from 0 to 1, sets IOMMU_CQH, IOMMU_CQT to 0 and clears IOMMU_CQCSR bits CMD_ILL, CMD_TO, CQMF, FENCE_W_IP. The CQ may take some time to be active following setting CQEN to 1 until the activation request completes. When the CQ is active, the CQON bit reads 1. When CQEN is changed from 1 to 0, the CQ may stay active until the instructions already fetched from the CQ are being processed and/or there

Table 28: IOMMU Command Queue Control and Status Register

				are outstanding implicit loads from the CQ. When the CQ turns off, the CQON bit reads 0, IOMMU_CQH is set to 0, IOMMU_CQT is set to 0 and the IOMMU_CQCSR bits CMD_ILL, CMD_TO, CQMF, FENCE_W_IP are set to 0. When the CQON bit reads 0, the IOMMU guarantees that no implicit memory accesses to the CQ are in-flight and the CQ will not generate new implicit loads to the queue memory.
17	BUSY	RO	0x0	A write to IOMMU_CQCSR may require the IOMMU to perform many operations that may not occur synchronously to the write. When a write is observed by IOMMU_CQCSR, the BUSY bit is set to 1. When the BUSY bit is 1, behavior of additional writes to the IOMMU_CQCSR is implementation defined. Some implementations may ignore the second write and others may perform the actions determined by the second write. Software must verify that the BUSY bit is 0 before writing to the IOMMU_CQCSR. An IOMMU that can complete controls synchronously may hardwire this bit to 0.
[31:18]	Reserved			

Table 28: IOMMU Command Queue Control and Status Register

When IOMMU detects an illegal command (unsupported CmdCode or illegal format), IOMMU has the following behavior:

- The hardware automatically stops fetching commands from the command queue
- The head pointer is not updated by the hardware which still points to the beginning of the illegal command.
- No error record is generated.
- IOMMU will not drain the command queue and will remain stalled until software intervention.

The software is expected to deal with the error and follow the sequence described below:

- Read the head pointer

- Software has two options:
  a. Software wants to re-start from an empty queue. It must then set the tail pointer to the head pointer.
  b. Software wants to overwrite the illegal command with a valid one and keep the other commands in the queue. It must not then modify the tail pointer.
- Re-enable the Command queue by writing 1 to `IOMMU_CQCSR.CMD_ILL`.

At this point, IOMMU will start working on the first command in the queue.

6.9 Fault Queue Base Register

This 64-bit register holds the Fault queue PPN (44-bit) and its size. The queue base address and size register is programmable (RW) as this queue can be located in system memory.

IOMMU Fault Queue Base Register (`IOMMU_FQB`)				
Register Offset		`0x28`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	`LOG2SZ-1`	WARL	`0x0`	The `LOG2SZ-1` field holds the number of error records in Fault Queue as a log to base 2 minus 1. A value of 0 indicates a queue of size 2 entries. If the FQ has 128 or fewer entries, then the base address of the queue is always aligned to a 4-KiB page address. If the FQ has more than 128 entries then the FQ base address must be naturally aligned to 2LOG2SZ x 32.
[9:4]	Reserved			
[31:10]	`PPN_LO`	WARL	`0x0`	Physical page number (low part) pointing to the base of the fault queue
[45:32]	`PPN_HI`	WARL	`0x0`	Physical page number (high part) pointing to the base of the fault queue
[63:46]	Reserved			

Table 29: IOMMU Fault Queue Base Register

6.10 Fault Queue Head Register

This 32-bit register holds the head pointer of the queue. This register is a read/write register. It is updated by host software to let IOMMU's hardware know when a new or several entries in the queue have been updated.

IOMMU Fault Queue Head Register (IOMMU_FQH)				
Register Offset		0x30		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	INDEX	WARL	0x0	Head Index (only LOG2SZ bits are writeable)

Table 30: IOMMU Fault Queue Head Register

6.11 Fault Queue Tail Register

This 32-bit register holds the tail pointer of the queue. This register is read-only as it is updated by IOMMU's hardware and is provided to software for information only.

IOMMU Fault Queue Tail Register (IOMMU_FQT)				
Register Offset		0x34		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	INDEX	RO	0x0	Tail Index (maintained by IOMMU)

Table 31: IOMMU Fault Queue Tail Register

6.12 Fault Queue Control and Status Register

This 32-bit RW register holds the following control bits:

IOMMU Fault Queue Control and Status Register (IOMMU_FQCSR)				
Register Offset		0x4C		
Bits	Field Name	Attr.	Rst.	Description
0	FQCSR_FQEN	RW	0x0	Fault queue enable. SW sets to 1 to request FQ to be enabled. The FQ is enabled when FQCSR_FQON reads 1. SW sets to 0 to request FQ to be disabled. The FQ is disabled when FQCSR_FQON reads 0.
1	FQCSR_FIE	RW	0x0	Fault queue interrupt-enable. When set to 1, enables generation of interrupts from FQ.
[7:2]	Reserved			
8	FQCSR_FQMF	RW1C	0x0	Fault Queue Memory Fault. The FQCSR_FQMF bit is set to 1 if the IOMMU encounters an access fault when storing a fault record to the fault-queue. The fault-record is discarded and no more fault records are generated until software clears FQCSR_FQMF bit by writing 1 to the bit. An interrupt is

Table 32: IOMMU Fault Queue Control and Status Register

				generated if enabled and not already pending (i.e., IOMMU_IPSR.FIP == 1). Note that this field is volatile and may be masked in IP-XACT.
9	FQCSR_FQOF	RW1C	0x0	Fault Queue Overflow. The FQCSR_FQOF bit is set to 1 if the IOMMU needs to queue a fault record but the fault-queue is full (IOMMU_FQT = IOMMU_FQH - 1), i.e., a fault-queue overflow. The fault-record is discarded and no more fault records are generated until software clears it by writing 1 to the bit. An interrupt is generated if not already pending (i.e., IOMMU_IPSR.FIP == 1). Note that this field is volatile and may be masked in IP-XACT.
[15:10]	Reserved			
16	FQCSR_FQON	RO	0x0	The fault-queue enable (FQCSR_FQEN) bit enables the fault-queue when set to 1. Changing FQCSR_FQEN from 0 to 1, resets the IOMMU_FQH and IOMMU_FQT to 0 and clears IOMMU_FQCSR bits FQCSR_FQMF and FQCSR_FQOF. The FQ may take some time to be active following setting the FQCSR_FQEN to 1. When the fault-queue is active, the FQCSR_FQON bit reads 1. Likewise when FQCSR_FQEN is changed from 1 to 0, the fault-queue may stay active until in-flight fault-recording is completed. When the fault-queue turns off, the FQCSR_FQON bit reads 0. The IOMMU guarantees that there are no in-flight implicit writes to the FQ in progress when FQCSR_FQON reads 0 and no new fault records will be written to the FQ.
17	FQCSR_BUSY	RO	0x0	A write to IOMMU_FQCSR may require the IOMMU to perform many operations that may not occur synchronously to the write. When a write is observed by the IOMMU_FQCSR, FQCSR_BUSY is set to 1. When FQCSR_BUSY is 1, behavior of additional writes to the IOMMU_FQCSR are implementation defined. Some implementations may ignore the second

Table 32: IOMMU Fault Queue Control and Status Register

				write and others may perform the actions determined by the second write. Software should ensure that FQCSR_BUSY is 0 before writing to the IOMMU_FQCSR. An IOMMU that can complete controls synchronously may hardwire this bit to 0.
[31:18]	Reserved			

Table 32: IOMMU Fault Queue Control and Status Register

6.13 Page-Request Queue Base Register

This 64-bit register holds the Page Request queue PPN (44-bit) and its size. The queue base address and size register is programmable (RW) as this queue can be located in system memory.

IOMMU Page-Request Queue Base Register (IOMMU_PQB)				
Register Offset		0x38		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	LOG2SZ-1	WARL	0x0	The LOG2SZ-1 field holds the number of error records in the Page-Request Queue as a log to base 2 minus 1. A value of 0 indicates a queue of size 2 entries. If the PQ has 256 or fewer entries then the base address of the queue is always aligned to a 4-KiB page address. If the PQ has more than 256 entries then the PQ base address must be naturally aligned to 2LOG2SZ x 16.
[9:4]	Reserved			
[31:10]	PPN_LO	WARL	0x0	Physical page number (low part) pointing to the base of the page request queue
[45:32]	PPN_HI	WARL	0x0	Physical page number (high part) pointing to the base of the page request queue
[63:46]	Reserved			

Table 33: IOMMU Page-Request Queue Base Register

6.14 Page-Request Queue Head Register

This 32-bit register holds the head pointer of the queue. This register is a read/write register. It is updated by host software to let IOMMU's hardware know when a new or several entries in the queue have been updated.

IOMMU Page-Request Queue Head Register (IOMMU_PQH)				
Register Offset		0x40		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	PQHEAD	WARL	0x0	Page request queue head index (only LOG2SZ bits are writeable)

Table 34: IOMMU Page-Request Queue Head Register

6.15 Page-Request Queue Tail Register

This 32-bit register holds the tail pointer of the queue. This register is read only as it is updated by IOMMU's hardware and is provided to software for information only.

IOMMU Page-Request Queue Tail Register (IOMMU_PQT)				
Register Offset		0x44		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	INDEX	RO	0x0	Tail Index (maintained by IOMMU)

Table 35: IOMMU Page-Request Queue Tail Register

6.16 Page-Request Queue Control and Status Register

This 32-bit RW register holds the following control bits:

IOMMU Page-Request Queue Control and Status Register (IOMMU_PQCSR)				
Register Offset		0x50		
Bits	Field Name	Attr.	Rst.	Description
0	PQEN	RW	0x0	Page-Request queue enable. SW sets to 1 to request PQ to be enabled. The PQ is enabled when PQON reads 1. SW sets to 0 to request PQ to be disabled. The PQ is disabled when PQON reads 0.
1	PIE	RW	0x0	The Page-Request interrupt-enable (PIE) bit when set to 1, enables generation of interrupts from PQ.
[7:2]	Reserved			
8	PQMF	RW1C	0x0	Page Request Queue Memory Fault. The PQMF bit is set to 1 if the IOMMU encounters an access fault when storing a Page Request record to the Page-Request queue. The Page-Request record is discarded and no more Page Request records are generated until software clears PQMF bit by writing 1 to the bit. An interrupt is

Table 36: IOMMU Page-Request Queue Control and Status Register

				generated if enabled and not already pending (i.e., IOMMU_IPSR.PIP == 1). Note that this field is volatile and may be masked in IP-XACT.
9	PQOF	RW1C	0x0	Page Request Queue Overflow. The PQOF bit is set to 1 if the IOMMU needs to queue a Page Request record but the Page-Request queue is full (IOMMU_PQT = IOMMU_PQH - 1), i.e., a Page-Request queue overflow. The Page-Request record is discarded and no more Page Request records are generated until software clears it by writing 1 to the bit. An interrupt is generated if not already pending (i.e., IOMMU_IPSR.PIP == 1). Note that this field is volatile and may be masked in IP-XACT.
[15:10]	Reserved			
16	PQON	RO	0x0	The Page-Request queue enable (PQEN) bit enables the Page-Request queue when set to 1. Changing PQEN from 0 to 1, resets the IOMMU_PQH and IOMMU_PQT to 0 and clears IOMMU_PQCSR bits PQMF and PQOF. The FQ may take some time to be active following setting the PQEN to 1. When the Page-Request queue is active, the PQON bit reads 1. Likewise, when PQEN is changed from 1 to 0, the Page-Request queue may stay active till in-flight Page-Request recording is completed. When the Page-Request queue turns off, the PQON bit reads 0. The IOMMU guarantees that there are no in-flight implicit writes to the PQ in progress when PQON reads 0 and no new Page Request records will be written to the PQ.
17	BUSY	RO	0x0	A write to IOMMU_PQCSR may require the IOMMU to perform many operations that may not occur synchronously to the write. When a write is observed by the IOMMU_PQCSR, BUSY is set to 1. When BUSY is 1, behavior of additional writes

Table 36: IOMMU Page-Request Queue Control and Status Register

				to the IOMMU_PQCSR are implementation defined. Some implementations may ignore the second write and others may perform the actions determined by the second write. Software should ensure that BUSY is 0 before writing to the IOMMU_PQCSR. An IOMMU that can complete controls synchronously may hardwire this bit to 0.
[31:18]	Reserved			

Table 36: IOMMU Page-Request Queue Control and Status Register

6.17 Translation-Request IOVA Register

The TREQIOVA is a 64-bit WARL register used to implement a translation-request interface for debug. This register is present when IOMMU_CAPABILITY.DEBUG == 1.

Translation-Request IOVA Register (TREQIOVA)				
Register Offset		0x258		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[11:0]	TR_REQ_IOVA_PGOFF	WARL	0x0	The IOVA page-offset
[31:12]	TR_REQ_IOVA_VPN_LOW	WARL	0x0	The IOVA virtual page number
[63:32]	TR_REQ_IOVA_VPN_HIGH	WARL	0x0	The IOVA virtual page number

Table 37: Translation-Request IOVA Register

6.18 Translation-Request Control Register

The TRREQCTL is a 64-bit WARL register used to implement a translation-request interface for debug. This register is present when IOMMU_CAPABILITY.DEBUG == 1.

Translation-Request Control Register (TRREQCTL)				
Register Offset		0x260		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
0	TR_REQ_CTL_GO	RWS	0x0	This bit is set to indicate a valid request has been set up in the TREQIOVA/TRREQCTL registers for the IOMMU to translate. The IOMMU indicates completion of the requested translation by clearing this bit to 0. On completion, the results of the translation are in TRRESP register. The TR_REQ_CTL_GO bit is a read-write-sticky (RWS) bit that once set cannot be cleared by writing the register.
1	TR_REQ_CTL_PRIV	WARL	0x0	When set to 1, the requests needs Privileged Mode access for this translation
2	TR_REQ_CTL_EXE	WARL	0x0	When set to 1, the request needs execute access for this translation
3	TR_REQ_CTL_RWN	WARL	0x0	When set to 1, the request only needs read-only access for this translation
[11:4]	Reserved			
[31:12]	TR_REQ_CTL_PID	WARL	0x0	When TR_REQ_CTL_PV is 1, this field provides the process_id for this translation request
32	TR_REQ_CTL_PV	WARL	0x0	When set to 1, the TR_REQ_CTL_PID field of the register is valid
[39:33]	Reserved			
[63:40]	TR_REQ_CTL_DID	WARL	0x0	This field provides the device_id for this translation request

Table 38: Translation-Request Control Register

6.19 Translation-Response Register

The TRRESP is a 64-bit RO register used to hold the results of a translation requested using the translation-request interface. This register is present when IOMMU_CAPABILITIES.DEBUG == 1.

Translation-Response Register (TRRESP)				
Register Offset		0x268		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
0	TR_RESPONSE_FAULT	RO	0x0	If the process to translate the IOVA detects a fault then the TR_RESPONSE_FAULT field is set to 1. The detected fault may be reported through the fault queue.
[6:1]	Reserved			
[8:7]	TR_RESPONSE_PBMT	RO	0x0	Memory type determined for the translation using PBMT fields in the S/VS-stage and/or the G-stage page tables used for the translation. The value of this field is UNSPECIFIED if TR_RESPONSE_FAULT is 1.
9	TR_RESPONSE_S	RO	0x0	Translation size field, when set to 1 indicates that the translation appies to a range that is larger than 4 KiB and the size of the translation range is encoded in the TR_RESPONSE_PPN field. The value of this field is UNSPECIFIED if TR_RESPONSE_FAULT is 1.
[31:10]	TR_RESPONSE_PPN_LOW	RO	0x0	If TR_RESPONSE_FAULT is 0, then this field provides the PPN (low bits) determined as a result of translating TREQIOVA.TR_REQ_IOVA_VPN. If TR_RESPONSE_FAULT is 1, then the value of this field is UNSPECIFIED. If TR_RESPONSE_S is 0, then the size of the translation is 4 KiB - a page. If TR_RESPONSE_S is 1, then the translation resulted in a super-page, and the size of the super-page is encoded in the PPN itself. If scanning from bit position 0 to bit position 43, the first bit with a value of 0 at position x, then the super-page size is $2^{x+1} * 4$ KiB. If x is not 0, then all bits at position 0 through x-1 are each encoded with a value of 1. See Table 40.

Table 39: Translation-Response Register

[53:32]	TR_RESPONSE_PPN_HIGH	RO	0x0	PPN (high bits). See description of TR_RESPONSE_PPN_LOW.
[63:54]	Reserved			

Table 39: Translation-Response Register

TR_RESPONSE_PPN	TR_RESPONSE_S	Size
yyyy....yyyy yyyy yyyy	0	4 KiB
yyyy....yyyy yyyy 0111	1	64 KiB
yyyy....yyy0 1111 1111	1	2 MiB
yyyy....yy01 1111 1111	1	4 MiB

Table 40: Example of Encoding of Super-Page Size in TR_RESPONSE_PPN

Translation-request registers that are used by software to program an IOVA and other inputs needed by the process to translate an IOVA as an Untranslated Request. The result of the translation, if the process completes successfully, is reported to through the translation-response registers. If the process stops due to faults then the faults are reported normally in the fault-queue and the translation-response registers are updated with a failure indicator.

When the process to translate an IOVA is invoked for this purpose, the IOMMU must not cache S/VS-stage PTEs, G-stage PTEs, DDT entries, or PDT entries. The IOMMU is allowed to use any PTEs or directory structure entries that may already be cached. When the IOMMU implements a HPM, the HPM counters may be updated normally by the IOMMU. For the purpose of counting in the HPM, these requests are treated as Untranslated Requests.

To request a translation, the TREQIOVA register is written first with the desired IOVA and the TRREQCTL register is written next. The TR_REQ_CTL_GO bit is set in TRREQCTL to indicate a valid request in the registers. The TR_REQ_CTL_GO bit is a read-write-sticky (RWS) bit that once set cannot be cleared by writing the register. The TR_REQ_CTL_GO bit will be cleared to 0 by the IOMMU when the process completes (successfully or due to encountering a fault). When the TR_REQ_CTL_GO bit goes from 1 to 0, it indicates that a response is valid in the TRRESP register.

The IOMMU behavior is UNSPECIFIED if:

- The TREQIOVA or TRREQCTL are modified when the TR_REQ_CTL_GO bit is 1
- IOMMU configurations such as IOMMU_DDTP.DDTP_IOMMU_MODE, etc. are modified

The time to complete a translation request through this debug interface is UNSPECIFIED but is required to be finite.

6.20 Interrupt Pending Status Register

The Interrupt-Pending Status Register reflects which units are requesting service.

IOMMU Interrupt Pending Status Register (IOMMU_IPSR)				
Register Offset		0x54		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
0	CIP	RW1C	0x0	Command queue interrupt-pending. Indicates an interrupt is pending software service. Software should clear this bit by writing 1. Note that this field is volatile and may be masked in IP-XACT.
1	FIP	RW1C	0x0	Fault queue interrupt-pending. Indicates an interrupt is pending software service. Software should clear this bit by writing 1. Note that this field is volatile and may be masked in IP-XACT.
2	PMIP	RW1C	0x0	Performance monitor interrupt-pending. Indicates an interrupt is pending software service. Software should clear this bit by writing 1. Note that this field is volatile and may be masked in IP-XACT.
3	PIP	RW1C	0x0	Page-request queue interrupt-pending. Indicates an interrupt is pending software service. Software should clear this bit by writing 1. Note that this field is volatile and may be masked in IP-XACT.
[7:4]	Reserved			

Table 41: IOMMU Interrupt Pending Status Register

6.21 Interrupt Cause Vector Register

Interrupt-cause-to-vector register maps a cause to a vector. All causes can be mapped to the same vector or a cause can be given a unique vector. The vector is used:

- For MSI capable IOMMU - index into MSI table
- For non-MSI capable IOMMU drive a wire.
  If an implementation only supports single vector then all bits of this register can be made hardwired to 0. Likewise if only two vectors are supported then only bit 0 for each cause could be RW.

IOMMU Interrupt Cause Vector Register (IOMMU_ICVEC)				
Register Offset		0x2F8		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[1:0]	ICVEC_CIV	WARL	0x0	The vector number assigned to the command queue interrupt. The vector number is used to index into the MSI table for a MSI-capable IOMMU to determine the MSI to generate. The vector number determines the wires to use to signal interrupt for a wired interrupt based IOMMU.
[3:2]	Reserved			
[5:4]	ICVEC_FIV	WARL	0x0	The vector number assigned to the fault queue interrupt. The vector number is used to index into the MSI table for a MSI-capable IOMMU to determine the MSI to generate. The vector number determines the wires to use to signal interrupt for a wired interrupt based IOMMU.
[7:6]	Reserved			
[9:8]	ICVEC_PMIV	WARL	0x0	The vector number assigned to the performance monitor interrupt. The vector number is used to index into the MSI table for a MSI-capable IOMMU to determine the MSI to generate. The vector number determines the wires to use to signal interrupt for a wired interrupt based IOMMU.
[11:10]	Reserved			
[13:12]	ICVEC_PIV	WARL	0x0	The vector number assigned to the page-request queue interrupt. The vector number is used to index into the MSI table for a MSI-capable IOMMU to determine the MSI to generate. The vector number determines the wires to use to signal interrupt for a wired interrupt based IOMMU.
[15:14]	Reserved			

Table 42: IOMMU Interrupt Cause Vector Register

Note

This register is hardwired to 0 if MSI generation is not supported and if a single wire interrupt is used.

6.22 MSI Configuration Table

IOMMU that supports MSI implements a MSI table that is indexed by the vector from `IOMMU_ICVEC` to determine a MSI table entry. Each MSI table entry `N` has three registers `IOMMU_MSI_ADDRESS_N`, `IOMMU_MSI_DATA_N`, and `IOMMU_MSI_CTL_0`. These registers are hard-wired to 0 if the IOMMU does not support MSI.

The `IOMMU_MSI_ADDRESS_N` holds the 4-byte aligned MSI address for interrupt vector `N`. For example, for entry 0:

IOMMU MSI Address Register 0 (`IOMMU_MSI_ADDRESS_0`)				
Register Offset		`0x300`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[1:0]	Reserved			
[31:2]	`MSI_ADDR_LOW_0`	WARL	`0x0`	MSI address low bits for entry 0
[47:32]	`MSI_ADDR_HIGH_0`	WARL	`0x0`	MSI address high bits for entry 0
[63:48]	Reserved			

Table 43: IOMMU MSI Address Register 0

The `IOMMU_MSI_DATA_N` holds the 4-byte MSI data for interrupt vector ‘N’. For example, for entry 0:

IOMMU MSI Data Register 0 (`IOMMU_MSI_DATA_0`)				
Register Offset		`0x308`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[31:0]	`MSI_DATA_0`	RW	`0x0`	MSI data for entry 0

Table 44: IOMMU MSI Data Register 0

The `IOMMU_MSI_CTL_N` holds the per-vector mask bit - `M`. While a vector is masked, the IOMMU is prohibited from sending the associated message. For example, for entry 0:

IOMMU MSI Control Register 0 (IOMMU_MSI_CTL_0)				
Register Offset		0x30C		
Bits	Field Name	Attr.	Rst.	Description
0	MSI_MASK_0	RW	0x0	MSI mask for entry 0
[7:1]	Reserved			

Table 45: IOMMU MSI Control Register 0

Note

These registers are hardcoded to 0 when wire interrupt is supported.

If an access fault is detected on a MSI write using IOMMU_MSI_ADDRESS_N, then the IOMMU reports a "IOMMU MSI write access fault" (cause 273) fault, with TTYP set to 0 and iotval set to the value of IOMMU_MSI_ADDRESS_N.

6.23 Version ID Register

Version ID Register (VERSIONID)				
Register Offset		0x2B0		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	RTL_VERSION	RO	0x101	IOMMU RTL Version
[31:16]	IMPID	RO	0xE24	IOMMU Implementation ID

Table 46: Version ID Register

VERSIONID Value	Release Version
0x0E240100	24G1.01.00
0x0E240101	24G1.02.00

Table 47: Log of VERSIONID Values

6.24 IOMMU DTI-ATS Status Register

IOMMU DTI-ATS Status Register (IOMMU_DTISR)				
Register Offset		0x2B4		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[1:0]	DTI_ATS_STATE	RO	0x0	DTI-ATS channel state: • 2'b00: DISCONNECTED state • 2'b01: REQ_CONNECT state • 2'b10: CONNECTED state • 2'b11: REQ_DISCONNECT state
2	NOTRANS	RO	0x0	NO_TRANS value extracted from the last connection request (DTI_ATS_CONDIS_REQ)
3	Reserved			
4	FENCE_PENDING	RO	0x0	When set, an IOFENCE.C command is under execution in the ATS module waiting for a DTI_SYNC_ACK response
5	PAGEACKTOSEND	RO	0x0	When set, a DTI_ATS_PAGE_ACK is pending to be sent upstream after reception of a DTI_ATS_PAGE_REQ
6	RESPACK_PENDING	RO	0x0	When set, a DTI_ATS_PAGE_RESP has been sent upstream and the ATS module is waiting for a DTI_ATS_RESPACK
7	Reserved			
[13:8]	TOKREQ	RO	0x0	Current number of translation tokens available
[15:14]	Reserved			
[21:16]	TOKINV	RO	0x0	Current number of invalidation tokens available
[31:22]	Reserved			

Table 48: IOMMU DTI-ATS Status Register

6.25 IOMMU ATS Timeout Countdown Register

IOMMU ATS Timeout Countdown Register (IOMMU_TIMER)				
Register Offset		0x2B8		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[31:0]	COUNTER_LOW	RO	All ones	Current value (low bits) of the down counter which triggers a timeout when it reaches 0
[37:32]	COUNTER_HIGH	RO	0x3F	Current value (high bits) of the down counter which triggers a timeout when it reaches 0
[62:38]	Reserved			
63	DEBUG_FAST_TIMEOUT	RW	0x0	Trigger a fast timeout for DV purpose. This feature is disabled when DBG_DISABLE is set.

Table 49: IOMMU ATS Timeout Countdown Register

6.26 Error Summary Status Register

Error Summary Status Register (SUMMARY_STATUS)				
Register Offset		0x1000		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	HIGHEST	RO	0x0	An encoding of the entry number that has the highest reported ErrorID. A zero means that there is no priorities reported by the input. Note that this field is volatile and may be masked in IP-XACT.
[7:4]	ENTRY1	RO	0x0	Entry field in the summary register, will be overwritten in the next cycle. Note that this field is volatile and may be masked in IP-XACT.
[11:8]	ENTRY2	RO	0x0	Entry field in the summary register, will be overwritten in the next cycle. Note that this field is volatile and may be masked in IP-XACT.
[15:12]	Reserved			

Table 50: Error Summary Status Register

ErrorID encoding:

- 0x0: None
- 0x1: Information, Event occurred - An event worth recording occurred.

- `0x2`: Corrected and cleaned, The error was corrected and the corrupted data was re-written or invalidated to avoid activating the same corruption again. A small perturbation in performance may be experienced due to the update activity.
- `0x3`: Corrected and uncleaned: The error was corrected before being used but the corrected data was not re-written to remove the corruption from the system automatically. The response agent should take action to avoid activating the same corruption again.
- `0x4`: Deferred and Produced, The error is uncorrected, but contained to an identifiable quantity that has been tainted and deferred to the consumer to determine severity and action.
- `0x5`: Deferred and Consumed, A tainted quantity has been consumed. The responding agent should access additional information to determine if this can be recovered.
- `0x6`: UnCorrected and Contained, The corruption is contained at the detecting component dropping the corrupted information and possibly preventing future outputs.
- `0x7`: UnCorrected and Uncontained, The corruption has not been contained and corrupted state may reach permanent storage unless enhanced responses are taken to contain the corruption.

Note

A parity error detected on the IOMMU Caches, even if it is an uncorrected error at the cache level, is considered Corrected and Cleaned because the IOMMU invalidates the line and it performs a page table walk to fetch the proper information.

6.27 Error Bank Revision/Personality Register

Error Bank Revision/Personality Register (`ERROR_REVISION_PERSONALITY`)				
Register Offset		`0x1008`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[7:0]	`REVISIONID`	RO	`0x10`	The revision identifier of the Error Bank. This is split into a 4b minor and a 4b major identifier.
[31:8]	Reserved			
[43:32]	`INSTANCEID`	RW	`0x0`	A unique identifier associated with this Error Bank. This value is represented in the InstanceID field of the Error Entry Status Register when an error event triggers an update of an entry's ErrorID field.
[63:44]	Reserved			

Table 51: Error Bank Revision/Personality Register

6.28 Error Entry Status Register

Error Entry Status Register (ERROR_STATUS)				
Register Offset		0x1010		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	INDEX	WARL	0x0	A unique error bank entry identifier. This is used to write specific bank entry status and supplemental registers. **Note:** Agents may re-purpose these values for tracking and annotation if they store the status values in a buffer. Note that this field is volatile and may be masked in IP-XACT.
[7:4]	INTERLOCK	WARL	0x0	The interlock value is changed when the status register is updated. This provides a write interlock and avoids overwriting new information. A write to the status register will fail unless the written index and interlock matches the current index and interlock field values for this error entry. **Note:** Agents may re-purpose these values for tracking and annotation if they store the status values in a buffer. Note that this field is volatile and may be masked in IP-XACT.
[10:8]	ERRORID	RW	0x0	ErrorID of the logged error event. Note that this field is volatile and may be masked in IP-XACT.
11	SUPPLEMENTALVALID	RW	0x0	Indicates that, whether the supplemental information in the corresponding supplemental register for the particular entry is valid. Note that this field is volatile and may be masked in IP-XACT.
[14:12]	LOSTERRORID	RW	0x0	An encoding of the error event's configured ErrorID that was overwritten. Note that this field is volatile and may be masked in IP-XACT.
15	LOSTSUPPLEMENTALVALID	RW	0x0	Indicates that the supplemental logging register is valid for the event that was overwritten/lost.

Table 52: Error Entry Status Register

				Always reads 0 if not implemented and ensuring that Lost supplemental values are not accessed by software. Note that this field is volatile and may be masked in IP-XACT.
[19:16]	STRUCTUREIDTYPE	RW	0x0	StructureIDType. Set to 0x3 for the IOMMU when reporting an error (SOC, Infrastructure). Note that this field is volatile and may be masked in IP-XACT.
[27:20]	STRUCTUREID	RW	0x0	The structure ID associated with the error event. Note that this field is volatile and may be masked in IP-XACT.
[31:28]	QUALIFIER	RW	0x0	Qualifier. Set to 0x0 when reporting an error. Note that this field is volatile and may be masked in IP-XACT.
[43:32]	INSTANCEID	RW	0x0	The instance identifier that is programmed into the personalization register of the Error Bank. Note that this field is volatile and may be masked in IP-XACT.
[63:44]	TIMESTAMP	RW	0x0	This is a sampling of a free running counter that is captured when the current ErrorID and related fields of the status register are updated. If the implemented counter has less than 20b, then the unimplemented bits are zero and not writeable. Note that this field is volatile and may be masked in IP-XACT.

Table 52: Error Entry Status Register

The IOMMU-22 implements 2 parity bits: one covering even bits and another one covering odd bits on the SRAM lines. Parity error detection leads to a cache line invalidation, a page table walk, and a refill. This is managed by the IOMMU hardware itself. The error detected is reported to the error bank: the StructureIDType is set to 0x3 for the IOMMU as it is an element of the SoC infrastructure. The upper two bits of StructureID identify the IOMMU function. The lower 6 bits identify the source of the error within the IOMMU:

- StructureID=193 (0xC1) when the error is detected in the IOMMU.L2TLB (SRAM)
- StructureID=194 (0xC2) when the error is detected in the IOMMU.PTW (SRAM)

No Error Record to the Error Queue is generated as the error is considered corrected and cleaned. Future versions of the IOMMU may add more `StructureID`.

6.29 Error Bank Supplemental 0 Register

Error Bank Supplemental 0 Register (`ERROR_SUPPLEMENTAL_0`)				
Register Offset		`0x1018`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	`INDEX`	RW	`0x0`	Unique identifier of the supplemental reg. Note that this field is volatile and may be masked in IP-XACT.
[7:4]	`INFORMATIONID`	RW	`0x0`	An encoded value showing how to interpret the remaining values of the register. Set to `0xF` (µArch) for the IOMMU-22 Error reporting. Note that this field is volatile and may be masked in IP-XACT.
[31:8]	`INFORMATIONLO`	RW	`0x0`	Lower bits of supplemental information field. This field is available for event specific logging and additional information supplementing the details of the event held in the status register. • Bits [11:8]: *Reserved* • Bits [31:12] - `PID`: Holds the `process_id`, if any Note that this field is volatile and may be masked in IP-XACT.
[63:32]	`INFORMATIONHI`	RW	`0x0`	Upper bits of supplemental information field. See description of `INFORMATIONLO`. • Bit 32 - `PV`: When set, the transaction carries a valid `process_id` • Bit 33 - `PRIV`: Privilege mode of the transaction (1 if supervisor) • Bits [39:34] - `TTYP`: Reports the inbound transaction type that encountered the error. If `TTYP==0`, the error happened on the execution of the command.

Table 53: Error Bank Supplemental 0 Register

				• Bits [63:40] - `DID`: Holds the `device_id` of the transaction Note that this field is volatile and may be masked in IP-XACT.

Table 53: Error Bank Supplemental 0 Register

6.30 Error Bank Event Configure Simple Modifier

Error Bank Simple Modifier Register (`CONFIG_SIMPLE_MODIFIER`)				
Register Offset		`0x1028`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[3:0]	`SIMPLEMOD0EVENTID`	RW	`0x0`	Event_id field of the Simple Modifier 0 to indicate which input to modify the error id
[6:4]	`SIMPLEMOD0ERRORID`	RW	`0x0`	Error_id field of the Simple Modifier 0 to indicate what is the overriding error id value for the given event id
7	`SIMPLEMOD0ENABLE`	RW	`0x0`	Enable field of the Simple Modifier 0
[11:8]	`SIMPLEMOD1EVENTID`	RW	`0x0`	Event_id field of the Simple Modifier 1 to indicate which input to modify the error id
[14:12]	`SIMPLEMOD1ERRORID`	RW	`0x0`	Error_id field of the Simple Modifier 1 to indicate what is the overriding error id value for the given event id
15	`SIMPLEMOD1ENABLE`	RW	`0x0`	Enable field of the Simple Modifier 1

Table 54: Error Bank Simple Modifier Register

Chapter 7

Error Records

The generic Error Record format is a 32B message that consists of a 12-bit Cause Code and many parameters to help identify the fault cause.

[255:194]	193	192	[191:128]	[127:96]	[95:64]	[63:40]	[39:34]	33	32	[31:12]	[11:0]
iotval2			iotval	RSVD	*custom*	DID	TTYP	PRIV	PV	PID	CAUSE

Table 55: Error Record Format

The CAUSE field is a code indicating the cause of the fault/event.

CAUSE	Description	Reported if tc.DTF==1?
1	Instruction access fault	No
4	Read address misaligned	No
5	Read access fault	No
6	Write/AMO address misaligned	No
7	Write/AMO access fault	No
12	Instruction page fault	No
13	Read page fault	No
15	Write/AMO page fault	No
20	Instruction guest page fault	No
21	Read guest-page fault	No
23	Write/AMO guest-page fault	No
256	All inbound transactions disallowed (ddtp.iommu_mode == Off)	Yes
257	DDT entry load access fault	Yes
258	DDT entry not valid	Yes
259	DDT entry misconfigured	Yes
260	Transaction type disallowed	No
265	PDT entry load access fault	No
266	PDT entry not valid	No
267	PDT entry misconfigured	No
268	DDT data corruption	No
269	PDT data corruption	No
272	Internal datapath error	No
273	IOMMU MSI write access fault	Yes
2048	Unsupported Page Size on ATS	No

Table 56: Table Fault Record CAUSE Field Encodings

The TTYP field reports inbound transaction type.

TTYP	Description
0	None. Fault not caused by an inbound transaction.
1	Untranslated read for execute transaction
2	Untranslated read transaction
3	Untranslated write/AMO transaction
4	Reserved
5	Translated read for execute transaction
6	Translated read transaction
7	Translated write/AMO transaction
8	PCIe ATS Translation Request
9	Message Request
10 - 31	Reserved
32 - 63	Reserved for custom use

Table 57: Table Fault Record TTYP Field Encodings

If the TTYP is a transaction with an IOVA then it is reported in iotval. If the TTYP is a message request then the message code is reported in iotval.

DID holds the device_id of the transaction. If PV is 0, then PID and PRIV are 0. If PV is 1, the PID holds a process_id of the transaction, and if the privilege of the transaction was Supervisor then PRIV bit is 1 else its 0. The DID, PV, `PID, and PRIV fields are 0 if TTYP is 0.

If the CAUSE is guest-page fault then the bits [63:2] of the guest physical address are reported into iotval2[63:2]. If bit 0 of iotval2 is 1, then guest-page-fault was caused by an implicit memory access for VS-stage address translation. If bit 0 of iotval2 is 1, and the implicit access was a write, then bit 1 is set to 1, else it is set to 0.

1. A guest page fault caused on G-stage page table due to a VS-stage translation iotval2 holds GPA[63:2], R/W depending on A/D update in iotval2[1] and iotval2[0] set to 1 to indicate VS-stage translation. iotval holds the original IOVA (GVA). Since the IOMMU is not getting the page offset, iotval2 = {GPA[63:12]; 10’b0; 1’b0; 1’b1} and iotval = {IOVA[63:12]; 12’b0}.
2. A guest page fault caused on G-stage page table due to the final GPA translation to SPA. iotval2 holds GPA[63:2], R/W depending on A/D update in iotval2[1] and iotval2[0] set to 0. iotval holds the original IOVA (GVA). Since the IOMMU is not getting the page offset, iotval2 = {GPA[63:12]; 10’b0; 1’b0; 1’b0} and iotval = {IOVA[63:12]; 12’b0}.

When the G-stage is active, the memory accesses for reading PDT entries to locate the Process-context are implicit memory accesses for VS-stage address translation. If a guest-page fault was caused by implicit memory access to read PDT entries, then the bit 0 of iotval2 is reported as 1 and the bit 1 as 0. Since the IOMMU is not getting the page offset, iotval2 = {GPA[63:12]; 10’b0; 1’b0; 1’b1} and iotval = {IOVA[63:12]; 12’b0}.

A guest-page fault caused on G-stage page table due to G-stage only translation (first stage translation mode is bare). `iotval2` holds `GPA[63:2]`, R/W depending on A/D update in `iotval2[1]` and `iotval2[0]` set to 0 `iotval` holds the original IOVA (GPA) Since the IOMMU is not getting the page offset, `iotval2 = {GPA[63:12]; 10'b0; 1'b0; 1'b1}` and `iotval = {IOVA[63:12]; 12'b0}`.

The IOMMU may be unable to report faults through the fault-queue due to error conditions such as the fault-queue being full or the IOMMU encountering access faults when attempting to access the queue memory. A memory-mapped fault control and status register (`fqcsr`) holds information about such faults. If the fault-queue full condition is detected the IOMMU sets a fault-queue full (`fqof`) bit in `fqcsr`. If the IOMMU encounters a fault in accessing the fault-queue memory, the IOMMU sets a fault-queue memory access fault (`fqmf`) bit in fqcsr. While either error bits are set in `fqcsr`, the IOMMU discards the record that led to the fault and all further fault records. When an error bit is in the fqcsr changes state from 0 to 1 or when a new fault record is produced in the fault-queue, fault interrupt pending (`fip`) bit is set in the `fqcsr`.

Fault records are always generated to the Fault Queue whenever an error as listed above is detected except when a page fault is detected on an ATS Translation Request. The following table describes the error reporting for each translation case:

Cause	ATS Translation Request
0	
1	CA
4, 6	
5, 7	CA
12	Success; No Fault record
13	Success; No Fault record
15	Success; No Fault record
20	Success; No Fault record
21	Success; No Fault record
23	Success; No Fault record
256 to 260	UR
265	CA
266	Success; No Fault record
267	CA
2048	UR

Table 58: Error Response Per Fault Cause in ATS Translation Request

7.1 Hardware Error (RAS)

IOMMU implements an Error Bank to collect, log and inject errors per the SiFive Error Architecture.

The subset of caches implemented in SRAM include two parity bits to protect each way: one parity bit covering the even bits of the way, and one parity bit covering the odd bits of the way.

In the case of error detection, the entire line is invalidated, the translation request is sent to the miss queue for the page to be walked again, and the refill to the cache will then happen with correct information. The error is then corrected and cleaned.

Chapter 8

Page Request Record

Page-request queue is an in-memory queue data structure used to report PCIe ATS “Page Request” messages to software. The base PPN of this in-memory queue and the size of the queue is configured into a memory-mapped register called page-request queue base (`IOMMU_PQB`).

The tail of the queue resides in a IOMMU controlled read-only memory-mapped register called `IOMMU_PQT`. The `IOMMU_PQT` register holds an index into the queue where the next page-request message will be written by the IOMMU. Subsequent to writing the message, the IOMMU advances the `IOMMU_PQT` by 1.

The head of the queue resides in a software controlled read/write memory-mapped register called `IOMMU_PQH`. The `IOMMU_PQH` register holds an index into the queue where the next page-request message will be received by software. Subsequent to processing the message(s) software advances the `IOMMU_PQH` by the count of the number of messages processed.

If `IOMMU_PQH == IOMMU_PQT`, the page-request queue is empty.

If `IOMMU_PQT == (IOMMU_PQH - 1)`, the page-request queue is full.

The IOMMU may be unable to report page-requests through the queue due to error conditions such as the queue being full or the IOMMU encountering access faults when attempting to access queue memory. A memory-mapped page-request queue control and status register (`IOMMU_PQCSR`) is used to hold information about such faults. If the queue full condition is detected, the IOMMU discards the message and may auto-complete the page-request as specified by the PCIe ATS specification. On a page queue full condition the page-request-queue overflow (`PQOF`) bit is set in `IOMMU_PQCSR`. If the IOMMU encountered a fault in accessing the queue memory, page-request-queue memory access fault (`PQMF`) bit in `IOMMU_PQCSR`. While either error bits are set in `IOMMU_PQCSR`, the IOMMU discards subsequent page-request messages and may auto-complete them using an IOMMU generated “Page Group Response” message as specified by PCIe ATS specifications.

When an error bit is in the `IOMMU_PQCSR` changes state from 0 to 1, or when a new message is produced in the queue, page-request-queue interrupt pending (`PIP`) bit is set in the `IOMMU_IPSR`.

[127:64]	[63:40]	[39:35]	34	33	32	[31:12]	[11:0]
Payload	DID	Reserved	EXEC	PRIV	PV	PID	Reserved

Table 59: Page Request Record Format

The DID field holds the requester ID from the message. The PID field is valid if PV is 1 and reports the PASID from the PASID TLP prefix of the message. PRIV is set to 0 if the message did not have a PASID TLP prefix, otherwise it holds the “Privilege Mode Requested” bit from the PASID TLP prefix. The EXEC bit is set to 0 if the message did not have a PASID TLP prefix, otherwise it reports the “Execute Requested” bit from the PASID TLP prefix. All other fields are set to 0. The payload of the “Page Request” message (bytes 0x08 through 0x0F of the message) is held in the PAYLOAD field.

Chapter 9

Interrupts

IOMMU provides one interrupt line with the following interrupt sources:

- Command interrupt: `IOMMU_IPSR.CIP` is set if any of following are TRUE:
  - `IOMMU_CQCSR.CMD_ILL == 1`: illegal command from command queue
  - `IOMMU_CQCSR.CQMF == 1`: memory access fault from command queue
  - `IOMMU_CQCSR.CMD_TO == 1`: command timeout
  - `IOMMU_CQCSR.FENCE_W_IP == 1`: if a wired interrupt due to fence was caused
- Fault interrupt: `IOMMU_IPSR.FIP` is set if any of following are TRUE:
  - `IOMMU_FQCSR.FQCSR_FQMF == 1`: memory access fault when attempting to write to fault queue
  - `IOMMU_FQCSR.FQCSR_FQOF == 1`: fault queue full
  - `IOMMU_FQH != IOMMU_FQT`
- PRI interrupt: `IOMMU_IPSR.PIP` is set if any of following are TRUE:
  - `IOMMU_PQCSR.PQMF == 1`: memory access fault when attempting to write to page-request queue
  - `IOMMU_PQCSR.PQOF == 1`: page-request queue full
  - `IOMMU_PQH != IOMMU_PQT`
- Peformance Monitor interrupt:
  - `IOMMU_IPSR.PMIP` is set if any of following are TRUE:
    - `IOCNTOVF != 1`: One or more counters overflowed
  - When `PMIP` goes from 0→1, causes an interrupt to be generated
- RAS:
  - The IOMMU outputs the ErrorID to be used by the next level of hierarchy as defined by SiFive Error Architecture. An interrupt is generated by ORing the ErrorID bits from the Error Bank integrated in IOMMU.

Chapter 10

Hardware Performance Monitor

Optionally, IOMMU implements a performance monitoring facility following SiFive's standard. The performance monitor include some counters with filtering option.

The hardware performance monitor includes at most 32 event counters. The first one, named `IOHPMMCYCLES`, counts the number of clock cycles. The other counter is `IOHPM_COUNTER_*`. The minimum configuration should implement 8 counters.

All counter registers are 64-bit RW/WARL registers and the counters are N-bit counters but at least 32-bit counters. The overflow is asserted when all N bits of the counter are 1.

The counter registers have an arbitrary value after IOMMU is reset, and can be written with a given value.

The event selector registers, `MHPMEVENT1` to `MHPMEVENT31`, are 64-bit RW registers that control which event causes the corresponding counter to increment.

Table 20 lists the Hardware Performance Monitor registers within the IOMMU control register space.

Note

Although up to 32 counters are supported by the architecture, 16 counters are implemented in IOMMU.

Note

Although 64b counters are supported by the architecture, 40b counters are implemented in IOMMU.

10.1 Performance Monitor Cycle Counter Register

This 64-bit register is a free running clock cycle counter. This register is RW. There is no associated MHPMEVENT0.

IOMMU Performance Monitor Cycle Counter Register (IOHPMMCYCLES)				
Register Offset		0x60		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[7:0]	MCYCLE_0	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[15:8]	MCYCLE_1	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[23:16]	MCYCLE_2	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[31:24]	MCYCLE_3	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[39:32]	MCYCLE_4	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[62:40]	Reserved			
63	OVERFLOW	RW	0x0	Overflow status or Interrupt disable. Note that this field is volatile and may be masked in IP-XACT.

Table 60: IOMMU Performance Monitor Cycle Counter Register

The OVERFLOW bit is set when the corresponding IOHPMMCYCLES counter overflows, and remains set until cleared by software. Note that there is no loss of information after an overflow since the counter wraps around and keeps counting while the sticky OVERFLOW bit remains set. If the IOHPMMCYCLES counter overflows when the associated OVERFLOW bit is zero, then an HPM Counter Overflow interrupt is generated by setting the IOMMU_IPSR.PMIP bit to 1. If the OVERFLOW bit is already one, then no interrupt request is generated. Consequently, the OVERFLOW bit also functions as a count overflow interrupt disable.

10.2 Performance Monitor Event Counter Register

These registers are 64-bit RW counter registers. For example, IOHPM_COUNTER_1:

IOMMU Performance Monitor Event Counter Register 1 (IOHPM_COUNTER_1)				
Register Offset		0x68		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[7:0]	COUNTER_1_0	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[15:8]	COUNTER_1_1	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[23:16]	COUNTER_1_2	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[31:24]	COUNTER_1_3	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[39:32]	COUNTER_1_4	WARL	0x0	Counter Value. Note that this field is volatile and may be masked in IP-XACT.
[63:40]	Reserved			

Table 61: IOMMU Performance Monitor Event Counter Register 1

10.3 Performance Monitor Event Selector Registers

These event registers are 64-bit RW registers. For example, for MHPMEVENT1:

IOMMU Performance Monitor Event Selector 1 Register (MHPMEVENT1)				
Register Offset		0x160		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
[14:0]	EVENT_ID_1	WARL	0x0	Specifies which event is to be counted in a filtered manner if not inhibited by fields DV_GSCV or PV_PSCV. • A value of 0 indicates no events are counted. • Encodings 1 to 16383 are reserved for standard events (Table 65) • Encodings 16384 to 32767 are for custom use and are implementation defined (Table 66) When EVENT_ID is changed, including to 0, the counter retains its value.
15	DMASK_1	RW	0x0	DMASK filtering. When set to 1, a partial matching of the DID_GSCID is performed for the transaction. The lower bits of the DID_GSCID all the way to the first low order 0 bit (including the 0 bit position itself) are masked.
[31:16]	PID_PSCID_LSB_1	RW	0x0	Counts events using this process_id (IDT==0) or PSCID (IDT==1)
[35:32]	PID_PSCID_MSB_1	RW	0x0	Counts events using this process_id (IDT==0) or PSCID (IDT==1)
[59:36]	DID_GSCID_1	RW	0x0	Counts events using this device_id (IDT==0) or GSCID (IDT==1)
60	PV_PSCV_1	RW	0x0	If set, only transactions with matching process_id or PSCID (based on the Filter ID Type) are counted
61	DV_GSCV_1	RW	0x0	If set, only transactions with matching device_id or GSCID (based on the Filter ID Type) are counted
62	IDT_1	RW	0x0	Filter ID Type: This field indicates the type of ID to filter on. When 0, the DID_GSCID field holds a device_id and the PID_PSCID field holds a process_id. When 1, the DID_GSCID field holds a GSCID and PID_PSCID field holds a PSCID.

Table 62: IOMMU Performance Monitor Event Selector 1 Register

63	OVERFLOW_1	RW	0x0	Overflow status or Interrupt disable. Note that this field is volatile and may be masked in IP-XACT.

Table 62: IOMMU Performance Monitor Event Selector 1 Register

DMASK	DID_GSCID	Comment
0	yyyyyyyy yyyyyyyy yyyyyyyy	One specific seg:bus:dev:func
1	yyyyyyyy yyyyyyyy yyyyy011	seg:bus:dev - any func
1	yyyyyyyy yyyyyyyy 01111111	seg:bus - any dev:func
1	yyyyyyyy 01111111 11111111	seg - any bus:dev:func

Table 63: Encoding of DMASK

The table below summarizes the filtering option of the counters:

IDT	DV_GSCV	PV_PSCV	Operation
0/1	0	0	Counter increments. No ID based filtering.
0	0	1	If the transaction has a valid process_id, counter increments if process_id matches PID_PSCID.
0	1	0	Counter incremented if device_id matches DID_GSCID.
0	1	1	If the transaction has a valid process_id, counter increments if device_id matches DID_GSCID and process_id matches PID_PSCID.
1	0	1	If the transaction has a valid PSCID, counter increments if the PSCID of that process matches PID_PSCID.
1	1	0	Counter incremented if GSCID is valid and matches DID_GSCID.
1	1	1	Counter increments if GSCID is valid and matches DID_GSCID and if PSCID is valid and matches PID_PSCID.

Table 64: Filtering Options

The OVERFLOW bit is set when the corresponding IOHPM_COUNTER_* overflows, and remains set until written by software. Since IOHPM_COUNTER_* values are unsigned values, overflow is defined as unsigned overflow. Note that there is no loss of information after an overflow since the counter wraps around and keeps counting while the sticky OVERFLOW bit remains set.

If an `IOHPM_COUNTER_*` overflows while the associated `OVERFLOW` bit is zero, then a HPM Counter Overflow interrupt is generated. If the `OVERFLOW` bit is one, then no interrupt request is generated. Consequently the `OVERFLOW` bit also functions as a count overflow interrupt disable for the associated `IOHPM_COUNTER_*`.

Note

There are not separate overflow status and overflow interrupt enable bits. In practice, enabling overflow interrupt generation (by clearing the `OVERFLOW` bit) is done in conjunction with initializing the counter to a starting value. Once a counter has overflowed, it and the `OVERFLOW` bit must be reinitialized before another overflow interrupt can be generated.

This HPM Counter Overflow interrupt (OR of all `IOHPM_COUNTER_*` overflows) is controlled and reported through `IOMMU_IPSR` register.

The Hardware Performance Monitor supports the following list of events to be counted:

Event Number	Event Description	`IDT` Filtering Available
0	Do not count	—
1	Counts the number of Untranslated requests at device interface	`IDT` == 0
2	Counts the number of Translated requests at device interface	`IDT` == 0
3	Counts the number of ATS Translation requests at device interface	`IDT` == 0
4	Counts the number of TLB miss	`IDT` == 0 or 1
5	Counts the number of Device Directory Walks	`IDT` == 0
6	Counts the number of Process Directory Walks	`IDT` == 0
7	Counts the number of S/VS-stage Page Table Walks	`IDT` == 0 or 1
8	Counts the number of G-stage Page Table Walks	`IDT` == 0 or 1

Table 65: Standard Events List

Event Number	Event Description	IDT
16384	Do not count	—
16385	Counts the number of clock cycles for DDT walk	IDT == 0
16386	Counts the number of clock cycles for PDT walk	IDT == 0
16387	Counts the number of clock cycles for S/VS-stage PT walk	IDT == 0 or 1
16388	Counts the number of clock cycles for G-stage PT walk	IDT == 0 or 1
16389	Counts the number of clock cycles where the Device Translation Request interface asserts the Ready signal Low	—
16390	Counts the number of clock cycles where the Device Translation Completion interface asserts the Ready signal Low	—

Table 66: Custom Events List

10.4 Performance Monitor Counter Inhibit Register

This Counter inhibit register is a 32-bit RW register.

Performance monitor counter inhibit register is able to inhibit selected counters from counting. Bits 31:0] inhibit counting in IOHPM_COUNTER 31-0 respectively, where counters 0 is known by its other name of IOHPMMCYCLES.

IOMMU Performance Monitor Counter Inhibit Register (IOCNTINH)				
Register Offset		0x5C		
Bits	Field Name	Attr.	Rst.	Description
0	MCYCLE_INHIBIT	RW	0x0	When set, IOHPMMCYCLES cycle counter is inhibited from counting
[15:1]	COUNT_INHIBIT	WARL	0x0	When set, the associated IOHPM_COUNTER.COUNTER event counter is inhibited from counting

Table 67: IOMMU Performance Monitor Counter Inhibit Register

This register allows precise programming in that one can inhibit counting, program multiple counters' event configurations, then unfreeze all the counters atomically via clearing IOCNTINH. Similarly, after the execution of interest, one can inhibit counting in IOCNTINH, and then sample the counter values, without worrying about the counters counting during the counter reads.

10.5 Performance Monitor Count Overflow Status Register

The Count Overflow status is a 32-bit read-only register that contains shadow copies of the OVERFLOW bits in the 31 MHPMEVENT* registers - where IOCNTOVF bit X corresponds to MHPMEVENTX and the OVERFLOW bit of IOHPMMCYCLES.

This register enables supervisor-level overflow interrupt handler software to quickly and easily determine which counter(s) have overflowed.

IOMMU Performance Monitor Count Overflow Register (`IOCNTOVF`)				
Register Offset		`0x58`		
Bits	**Field Name**	**Attr.**	**Rst.**	**Description**
0	`CYCLES_OVERFLOW`	RO	`0x0`	Cycles overflow (`IOHPMMCYCLES`) status. Note that this field is volatile and may be masked in IP-XACT.
[15:1]	`EVENTS_OVERFLOW`	RO	`0x0`	Events overflow (`MHPMEVENT*.OVERFLOW`) status. Note that this field is volatile and may be masked in IP-XACT.

Table 68: IOMMU Performance Monitor Count Overflow Register

Chapter 11

Debug

- A debug disable input is available; when set, debug functionality is disabled. This input can be connected to a fuse or a debug authentication signal.
- Use the register interface to perform address translation: `TREQIOVA`, `TRREQCTL`, and `TRRESP`.

Appendix A

Integration Example in a WorldGuard-Enabled Environment

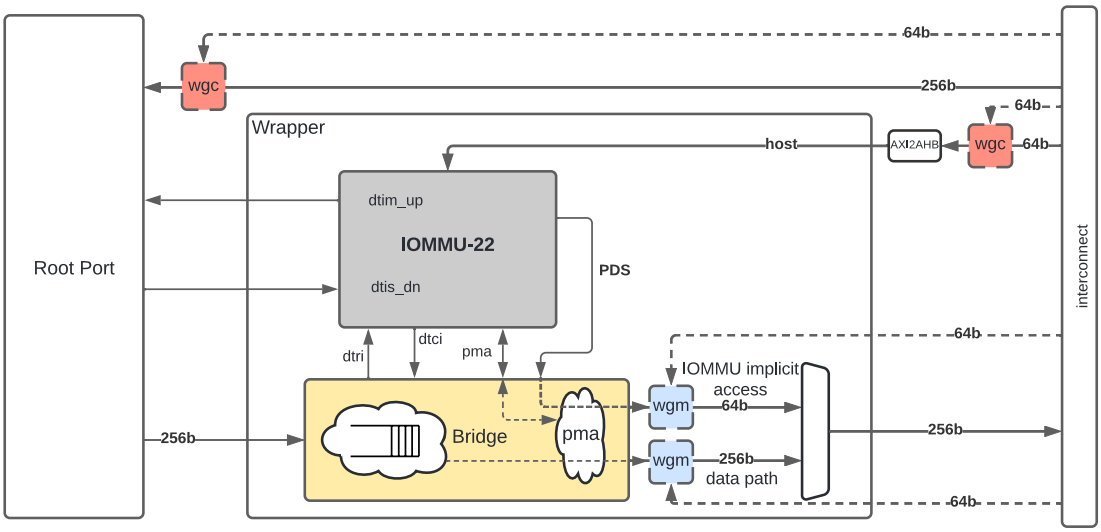


Figure 21: Integration Example in a WorldGuard-Enabled Environment

Similar to the RISC-V harts, physical memory attributes (PMA) and physical memory access marking done via WorldGuard must be completed on any inbound IO transactions even when the IOMMU is in bypass (bare state). The placement and integration of the PMA checkers and wgMarkers is a platform choice and reside outside the IOMMU. The example above is showing PMA checkers in the IO Bridge and wgMarkers in the wrapper.

Implicit accesses by the IOMMU itself through the data structure interface are checked by the PMA checker. PMAs are tightly tied to a given physical platform's organization, and many details are inherently platform-specific. The IOMMU provides the resolved PBMT (PMA, IO, NC) along with the translated address on the device translation completion interface to the IO Bridge. The PMA in IO Bridge may use the provided PBMT to override the PMA(s) for the associated memory pages.

The wgChecker may use the hardware ID of the bus master to determine physical memory access privileges. As the IOMMU itself is a bus master for its implicit accesses, the IOMMU hardware ID may be used by the wgMarker to select the appropriate access control rules.

When the IOMMU is integrated into a WG enabled environment, WG gaskets must be added into the IObridge or wrapper.

- A wgChecker (wgc) is added in front of the Host interface in order to protect the IOMMU MMIO space.
- A wgMarker (wgm) is added on the IOMMU implicit access port: all implicit IOMMU accesses must be marked with a WID (Access to queues, page tables, context information).
- A wgMarker (wgm) is added on the IO Bridge datapath master port: the inbound data flow must be marked with a WID determined by the DeviceID given by the IO device. It is thus left to FW to assign an IO or an Function within this IO to a specific WID.

If one IOMMU is used for several IO devices, a single wgMarker will be needed at the IO Bridge master port to cover all IO devices inbound flow.

Note

The IO device has to provide the DeviceID to the IO Bridge/IOMMU. This same DeviceID can be used to assign a WID once the IOMMU has provided the Physical address.

The IO Bridge/wrapper integrating the IOMMU-22 must support the following functions:

- Data management and reordering while IOVA is being translated by IOMMU-22
- PMA Checking on PDS and Master data path to system memory
- PMA handshake with ATS module if source_cxl is set
- Drain rd/wr on fence commands
- Marking of both IOMMU implicit accesses and IO Bridge data path with physical addresses

Appendix B

Revision History

Version	Date	Document Changes
24G1.02.00	April 5, 2024	• *Chapter 5*: Moved Programming Model chapter from User Guide • *Section 2.1.4*: Added note about IOMMU PDS AXI transactions not crossing a 64B boundary • Editorial fixes
24G1.01.00	February 19, 2024	• Initial release

Table 69: Revision Information

References

[1] A. Waterman and K. Asanovic, Eds., The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.2, June 2019. [Online]. Available: https://riscv.org/specifications/

[2] A. Waterman and K. Asanovic, Eds., The RISC-V Instruction Set Manual Volume II: Privileged Architecture, Version 1.11, June 2019. [Online]. Available: https://riscv.org/specifications/privileged-isa/

[3] IOMMU Task Group, RISC-V IOMMU Specification Document, Version 0.9, October 2022. [Online]. Available: https://github.com/riscv-non-isa/riscv-iommu/blob/main/riscv-iommu.pdf